Material Purchase and Planning

Business Intelligence and Data Warehouse Implementation

Course: Current Topics in Computer Science

Institution: Vietnamese-German University

Instructor: Dr. Phan Trong Nhan

Date: October 26, 2025

Group Members:

1. Pham Duc Manh - ID: 10422047
2. Nguyen Thao Vy - ID: 10421067
3. Luong Tieu Cuong - ID: 104220889
4. Nguyen Thien Nguyen - ID: 10422059
5. Nguyen Hoang Yen Ngoc - ID: 10422056

Contribution Percentages

No	Assessments	Contribution List
1	Topic investigation and data source analysis (1 point)	Yen Ngoc, Tieu Cuong
2	Problem statement (0.5 point)	Yen Ngoc, Thien Nguyen
3	Data warehouse design to meet your needs (1.5 points)	Duc Manh, Thao Vy
4	ETL pipeline implementation with incremental loading and (near) real-time (1.5 points)	Thao Vy, Yen Ngoc
5	At least one data mining/machine learning technique is used to serve your business analysis (1.5 points)	Duc Manh, Thien Nguyen
6	A BI dashboard to visualize your analysis (1.5 points)	Tieu Cuong
7	One use-case to show how your analysis supports business decision making (0.5 point)	Thien Nguyen
8	Final presentation (1 point)	All team
9	Report (1 point)	All team

Abstract

This report details an in-depth investigation into CompanyX's Material Purchase and Planning operations, aimed at enhancing procurement efficiency, optimizing inventory levels, and improving supply chain reliability. The analysis is founded upon the comprehensive aggregation, cleansing, and transformation of transactional data extracted from the company's Enterprise Resource Planning (ERP) and Material Requirements Planning (MRP) systems.

A core component of this work involves the design and implementation of a data warehouse specifically engineered to support incremental loading and near real-time data updates, ensuring timely and accurate insights. A robust ETL (Extract, Transform, Load) pipeline was deployed to standardize purchasing and inventory data, establishing a reliable single source of truth.

Applying data mining and machine learning techniques to improve the accuracy and efficiency of material procurement planning. Specifically, Time Series Forecasting (Prophet) is utilized to predict the monthly demand quantity per product, enabling data-driven estimation of future material needs. In parallel, Random Forest Regression is employed to optimize vendor selection by evaluating multiple performance criteria such as cost, lead time, and reliability. Together, these models aim to support informed purchasing decisions, reduce procurement costs, and strengthen overall supply chain performance.The critical findings are visualized via an interactive BI dashboard, focusing on granular Spend Analysis and Inventory Optimization metrics.

Finally, the report illustrates a concrete use-case demonstrating how these analytical outputs directly support key business decision-making, such as recalibrating Reorder Points (ROP) for strategic materials, thereby mitigating stock-out risks while maximizing working capital.

Table of Contents

Abbreviations 5
List of Tables 6
List of Figures 7
1. Introduction 8
1.1. Topic Investigation 8
1.2. Data Source Analysis 10
1.3. Problem Statement 19
2. Data Warehouse 21
2.1. Model Design 21
2.1.1. Overview 21
2.1.2. Schema Structure 22
2.2. ETL Pipeline Implementation 28
2.2.1. ETL Overview 28
2.2.2. Data Transformation 37
2.2.3. Data Loading 43
3. Data Analytics 46
3.1. Techniques 46
3.1.1. Demand Forecasting Using Prophet 46
3.1.2. Vendor Selection Using Random Forest Regression 48
3.2. BI System 51
3.2.1. Dashboard Design 51
3.2.2. System Implementation 53
3.3. Case Study Supports Decision Making 55
3.3.1. Business Context and Challenge 55
3.3.2. Analytical Process: 55
3.3.3. Vendor Selection: 55
3.3.4. Decision-Making Process: 56
3.3.5. Business Impact 56
4. Conclusion 58

Abbreviations

Enterprise Resource Planning (ERP)

Material Requirements Planning (MRP)

Extract, Transform, Load (ETL)

Reorder Points (ROP)

Primary Key (PK)

Foreign Key (FK)

Entity-Relationship Diagram (ERD)

List of Tables

1. Introduction....8
Table 1. Description of Purchasing. Tables.... 10
Table 2. Description of Purchasing.PurchaseOrder Tables.... 11
Table 3. Description of Production. Tables.... 11
Table 4. Description of Production.ProductInventory Table.... 11
Table 5. Vendor Data: Master Record for Supplier Analysis.... 12
Table 6. Product-Vendor Link: Benchmarking Negotiated Pricing.... 13
Table 7. Ship Method: Logistics Data for Lead Time and Cost.... 13
Table 8. PO Header: Transactional Data for Order Tracking.... 14
Table 9. PO Detail: Granular Data for Spend Calculation.... 15
Table 10. Product Master: Universal Record for Material Classification.... 17
Table 11. Bill of Materials (BOM): Structural Data for Material Requirements.... 17
Table 12. Inventory Data: Dynamic Stock Levels for Optimization.... 18
2. Data Warehouse.... 19
Table 15: DimVendor Breakdown....23
Table 16: DimShipmethod Breakdown....23
Table 17: DimDate Breakdown.... 25
Table 18. ProductVendor Breakdown....26
Table 19: Key Transformation Principles....34
3. Data Analytics.... 42
Table 20. Derived Variables for Monthly Product Demand Forecasting....42
4. Conclusion....58

List of Figures

1. Introduction....8
Figure 1. ERD Relationships Between Tables.... 12
2. Data Warehouse.... 21
Figure 2. Data Warehouse Design....21
Figure 3: Choose the source column.... 31
Figure 4: Choose the source column.... 35
Figure 5: FactMaterial Data Flow....37
Figure 6: DimProduct Dataflow.... 38
Figure 7: DimVendor Dataflow.... 39
Figure 8: DimShipMethod Dataflow.... 40
Figure 9: DimDate Dataflow....41
Figure 10: ProductVendor (Bridge Table) Dataflow....42
Figure 11: Add New schedule task....46
3. Data Analytics.... 47
Figure 12. Aggregated Monthly Product Demand Data Used for Prophet Forecasting.... 47
Figure 13: Overview Dashboard for Material Purchase and Planning BI....51
Figure 14: BI Dashboard when selecting specific Product “Flat Washer 4”.... 53
4. Conclusion....58

1. Introduction

1.1. Topic Investigation

1.1.1. Business Context

CompanyX is recognized as a manufacturer and distributor of precision-engineered bicycle components, encompassing critical parts such as crankarms, bearings, chainrings, bolts, and structural tubes. Operating within a highly competitive global manufacturing ecosystem, the company's sustained success is intrinsically linked to the efficiency of its supply chain, the quality of procured materials (steel, aluminum, composite elements), and the seamless synchronization between procurement, inventory management, and production scheduling. While CompanyX maintains a strong market position, its current traditional purchasing and planning methods—which rely heavily on historical demand patterns and manual oversight—are no longer sustainable for agile global operations. This necessitates a strategic shift towards integrated, data-driven decision-making to maintain competitive advantage.

1.1.2. Current Operational Challenges

Despite its robust market standing, CompanyX is currently experiencing significant operational inefficiencies that negatively impact production continuity and financial performance.

A. Core Issue: Material Availability Mismatch

A critical and recurring event is the frequent mismatch between material availability and immediate production requirements. This manifests in two costly extremes: significant overstocking of lower-demand components, leading to high holding costs and capital lock-up, and simultaneous stockouts of high-demand, high-value parts such as crankarms and bearings, directly causing production delays and lost sales opportunities.

B. Root Causes of Inefficiency

The underlying reasons for these stock imbalances and related inefficiencies are systemic:

Data Fragmentation: Procurement and inventory data are often stored in disparate, disconnected systems, resulting in poor data visibility across Purchasing, Planning, and Production departments. This prevents a unified view of material flow and expenditure.

Manual Forecasting Vulnerability: The reliance on predominantly manual forecasting processes introduces inherent human error and lacks the statistical rigor required to model complex market variability and product seasonality. This results in systematically inaccurate demand planning.

Uncoordinated Supplier Management: The absence of a standardized, data-driven system for tracking supplier performance leads to inconsistent and often uncoordinated supplier management. This instability contributes directly to unexpected material delivery delays and inconsistent quality standards.

Absence of Centralized Analytics: The lack of a unified analytical platform prevents the early detection of performance bottlenecks and operational inefficiencies, limiting the company's ability to respond quickly to changes in market demand or supply chain disruptions.

Collectively, these factors escalate inventory holding costs, introduce delays into the production pipeline, and diminish CompanyX's overall responsiveness to a dynamic market.

1.1.3. Project Objectives

This project aims to directly resolve these structural challenges by implementing a comprehensive, data-driven Material Purchase and Planning System. The primary objectives are to enhance visibility, improve forecasting accuracy, and instill greater control across all relevant operations:

1/ Automated Demand Forecasting: To overcome inaccuracies from manual methods, the project will implement advanced analytical models utilizing historical production data and trend analysis to generate statistically robust forecasts, thereby significantly reducing stock imbalances.

2/ Data Centralization and Collaboration: Establishing a unified analytical platform (BI Dashboard) is crucial for centralizing procurement and inventory data. This will enable real-time monitoring and foster crucial cross-departmental collaboration, breaking down existing data silos.

3/ Strengthening Supplier Reliability: The project will formalize and automate supplier performance tracking, focusing on key metrics like Average Lead Time (vendor delivery time performance), Standard Cost and Last Receipt Cost. This structured approach will support strategic relationship building and ensure reliable material input.

4/ Optimizing Purchasing Strategy: Data insights will be leveraged to optimize purchasing decisions, allowing CompanyX to effectively balance the trade-off between material cost, required quality standards, and immediate material availability, moving beyond simple lowest-cost acquisition.

1.1.4. Project Success Metrics

The success of this implementation will be objectively measured against specific, quantifiable operational and financial targets:

1/ Inventory Turnover Rate: A targeted increase of at least 15%, serving as a key indicator of improved material flow efficiency and a reduction in capital tied up in excess stock.

2/ Material Availability Rate: Sustaining a rate of $\geq$ 90% material readiness for production, directly minimizing expensive production downtime.

3/ Procurement Cycle Time: Achieving a 20% reduction in the average lead time from purchase initiation to material receipt, driven by automation and proactive supplier tracking.

4/ Cost Efficiency: Realizing a measurable 5-10% annual reduction in total material spend, achieved through optimized purchasing strategies and lower material carrying costs.

5/ System Integration: Full and successful deployment of the centralized BI dashboard, ensuring automated, accurate, and consistent reporting on all critical Purchasing, Inventory, and Supplier KPIs.

1.2. Data Source Analysis

This section systematically outlines the structure and inherent characteristics of the source data tables relevant to Material Purchase and Planning at CompanyX. The analyzed data has been extracted from the company's core ERP platform and represents the transactional and master data across the Procurement and Production functional modules. This analysis serves to validate the data structure against the project's analytical requirements and anticipate potential data quality challenges before ETL development.

1.2.1. System & Data Landscape Overview

The analytical foundation is built upon a set of relational tables primarily categorized under the Purchasing and Production schemas. These tables collectively capture the end-to-end lifecycle, from vendor identification and purchase order execution to component bill-of-materials and final inventory levels.

The dataset is segmented into four primary analytical domains:

1/ Supplier & Procurement Master Data:

Tables Purchasing.Vendor, Purchasing.ProductVendor and Purchasing.ShipMethod provides the foundational static data required for assessing supplier reliability and modeling the cost impact of different shipping methods.

Purchasing.Vendor	Companies from whom Adventure Works Cycles purchases parts or other goods.
Purchasing.ProductVendor	Cross-reference table mapping vendors with the products they supply.
Purchasing.ShipMethod	Shipping company lookup table.

Table 1. Description of Purchasing. Tables

2/ Transactional Purchasing Data:

Purchasing.PurchaseOrderHeader and Purchasing.PurchaseOrderDetail provide the granular detail necessary for spend analysis (quantity, unit price) and tracking logistical execution (order dates, shipping methods).

Purchasing.PurchaseOrderHeader	General purchase order information.
Purchasing.PurchaseOrderDetail	Individual products associated with a specific purchase order.

Table 2. Description of Purchasing.PurchaseOrder Tables

3/ Product & Structural Data:

Production.Product - classifies all materials and finished goods and Production.BillsofMaterials serves as the structural translation layer, explicitly linking finished goods to their required raw material components. This linkage is essential for converting historical purchasing volumes into inferred demand signals for raw materials.

Production.Product	Describes all products manufactured or sold by CompanyX, including name, category, and cost.
Production.BillsofMaterials	Lists components or raw materials required to manufacture a product.

Table 3. Description of Production. Tables

4/ Inventory Data:

Production.ProductInventory records the current physical quantity on hand.

Production.ProductInventory	Tracks current stock levels of products in various locations.

Table 4. Description of Production.ProductInventory Table

The following Entity-Relationship Diagram (ERD) illustrates the complex dependencies between the core purchasing, inventory, and production tables. Understanding this native structure is crucial, as it defines the necessary join paths for data extraction (ETL) and highlights the inherent challenges of combining fragmented transactional and master data to derive unified analytical insights.

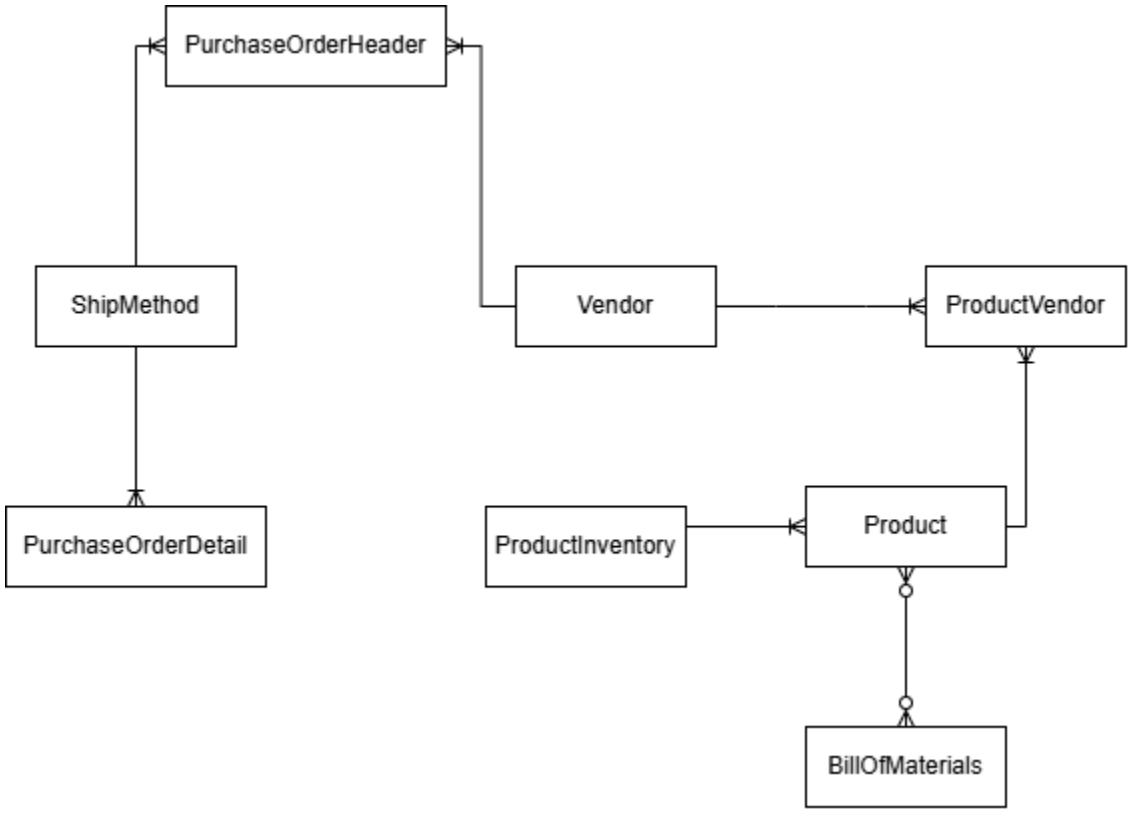


Figure 1. ERD Relationships Between Tables

The data schema reveals a highly interconnected structure that mirrors the end-to-end procurement and inventory lifecycle. The core transactional flow originates from **PurchaseOrderHeader**, which links uniquely to a single **Vendor** and a **ShipMethod**, establishing who is supplying and how the goods will arrive. This header then branches into multiple lines via **PurchaseOrderDetail**, detailing the specific items. Crucially, the relationship between products and suppliers is managed by the **ProductVendor** bridge table, enabling a many-to-many link: one **Product** can be sourced from multiple vendors, and one **Vendor** can supply many products. This mapping is essential for vendor evaluation.

The **Product** entity is central, connecting to **ProductInventory** to show current stock levels, and recursively linking through **BillOfMaterials**. The BOM structure defines how one **Product (as an assembly)** requires a specific component **Product (as materials),** which directly informs our planning needs by detailing the inherent dependencies in manufacturing. In summary, the connections are designed to track **Order → Vendor/Logistics → Product Details → Inventory Status**, while the BOM explicitly defines the structural requirements that drive future purchase decisions.

1.2.2. Detailed Component Composition (Schema Deep Dive)

To translate the conceptual data structure into a functional analytical model, this section provides a detailed decomposition of the core data tables. The focus is on identifying key attributes and linkages that are indispensable for calculating analytical metrics, such as Total Spend, Inventory Turnover Rate, and Material Requirements, thereby underpinning the subsequent BI dashboard development.

A/ Purchasing.Vendor

Attributes	Data Type	Key Linkage/ Nullable	Description
BusinessEntityID	int	PK, FK/ Not null	Primary key for **Vendor** records. Foreign key to **BusinessEntity.BusinessEntityID**
AccountNumber	nvarchar(15)	Not null	Vendor account (identification) number.
Name	nvarchar(50)	Not null	Company name.
CreditRating	tinyint	Not null	1 = Superior, 2 = Excellent, 3 = Above average, 4 = Average, 5 = Below average
PreferredVendorStatus	bit	Not null	0 = Do not use if another vendor is available. 1 = Preferred over other vendors supplying the same product.
ActiveFlag	bit	Not null	0 = Vendor no longer used. 1 = Vendor is actively used.
PurchasingWebServiceURL	nvarchar(1024)		Vendor URL.
ModifiedDate	datetime	Not null	Date and time the record was last updated.

Table 5. Vendor Data: Master Record for Supplier Analysis

B/ Purchasing.ProductVendor

Attributes	Data Type	Key Linkage/ Nullable	Description
ProductID	int	PK, FK/ Not	Primary key. Foreign key to

		null	**Product.ProductID**.
BusinessEntityID	int	PK, FK/ Not null	Primary key. Foreign key to **Vendor.BusinessEntityID**.
AverageLeadTime	int	Not null	The average span of time (in days) between placing an order with the vendor and receiving the purchased product.
StandardPrice	money	Not null	The vendor's usual selling price.
LastReceiptCost	money		The selling price when last purchased.
LastReceiptDate	datetime		Date the product was last received by the vendor.
MinOrderQty	int	Not null	The minimum quantity that should be ordered.
MaxOrderQty	int	Not null	The maximum quantity that should be ordered.
OnOrderQty	int		The quantity currently on order.
UnitMeasureCode	datetime	FK/ Not null	The product's unit of measure.
ModifiedDate	datetime	Not null	Date and time the record was last updated.

Table 6. Product-Vendor Link: Benchmarking Negotiated Pricing

C/ Purchasing.ShipMethod

Attributes	Data Type	Key Linkage/ Nullable	Description
ShipMethodID	int	PK/ Not null	Primary key for ShipMethod records.
Name	nvarchar(50)	Not null	Shipping company name.
ShipBase	money	Not null	Minimum shipping charge.
ShipRate	money	Not null	Shipping charge per pound.
rowguid	uniqueidentifier	Not null	ROWGUIDCOL number uniquely identifying

			the record. Used to support a merge replication sample.
ModifiedDate	datetime	Not null	Date and time the record was last updated.

Table 7. Ship Method: Logistics Data for Lead Time and Cost

D/ Purchasing.PurchaseOrderHeader

Attributes	Data Type	Key Linkage/ Nullable	Description
PurchaseOrderID	int	PK/ Not null	Primary key.
RevisionNumber	tinyint	Not null	Incremental number to track changes to the purchase order over time.
Status	tinyint	Not null	Order current status. 1 = Pending; 2 = Approved; 3 = Rejected; 4 = Complete
EmployeeID	int	FK/ Not null	Employee who created the purchase order. Foreign key to **Employee.BusinessEntityID**.
VendorID	int	FK/ Not null	Vendor with whom the purchase order is placed. Foreign key to **Vendor.BusinessEntityID**.
ShipMethodID	int	FK/ Not null	Shipping method. Foreign key to **ShipMethod.ShipMethodID**.
OrderDate	datetime	Not null	Purchase order creation date.
ShipDate	datetime		Estimated shipment date from the vendor.
SubTotal	money	Not null	Purchase order subtotal. Computed as **SUM (PurchaseOrderDetail.LineTotal)** for the appropriate **PurchaseOrderID**.
TaxAmt	money	Not null	Tax amount.
Freight	money	Not null	Shipping cost.
TotalDue (computed)	money	Not null	Total due to the vendor. Computed as Subtotal + TaxAmt + Freight.
ModifiedDate	datetime	Not null	Date and time the record was last updated.

Table 8. PO Header: Transactional Data for Order Tracking

E/ Purchasing.PurchaseOrderDetail

Attributes	Data Type	Key Linkage/ Nullable	Description
PurchaseOrderID	int	PK, FK/ Not null	Primary key. Foreign key to **PurchaseOrderHeader.PurchaseOrderID**.
PurchaseOrderDetailID	int	PK/ Not null	Primary key. One line number per purchased product.
DueDate	datetime	Not null	Date the product is expected to be received.
OrderQty	smallint	Not null	Quantity ordered.
ProductID	int	FK/ Not null	Product identification number. Foreign key to **Product.ProductID**.
UnitPrice	money	Not null	Vendor's selling price of a single product.
LineTotal (computed)	money	Not null	Per product subtotal. Computed as OrderQty * UnitPrice.
ReceivedQty	decimal(8,2)	Not null	Quantity actually received from the vendor.
RejectedQty	decimal(8,2)	Not null	Quantity rejected during inspection.
StockedQty (computed)	decimal(9,2)	Not null	Quantity accepted into inventory. Computed as ReceivedQty - RejectedQty.
ModifiedDate	datetime	Not null	Date and time the record was last updated.

Table 9. PO Detail: Granular Data for Spend Calculation

F/ Production.Product

Attributes	Data Type	Key Linkage/ Nullable	Description

ProductID	int	PK/ Not null	Primary key for **Product** records.
Name	nvarchar(50)	Not null	Name of the product.
ProductNumber	nvarchar(25)	Not null	Unique product identification number.
MakeFlag	bit	Not null	0 = Product is purchased, 1 = Product is manufactured in-house.
FinishedGoodsFlag	bit	Not null	0 = Product is not a salable item. 1 = Product is salable.
Color	nvarchar(15)		Product color.
SafetyStockLevel	smallint	Not null	Minimum inventory quantity.
ReorderPoint	smallint	Not null	Inventory level that triggers a purchase order or work order.
StandardCost	money	Not null	Standard cost of the product.
ListPrice	money	Not null	Selling price.
Size	nvarchar(5)		Product size.
SizetUnitMeasureCode	nchar(3)	FK	Unit of measure for Size column.
WeightUnitMeasureCode	nchar(3)	FK	Unit of measure for Weight column.
Weight	decimal(8,2)		Product weight.
DaysToManufacture	int	Not null	Number of days required to manufacture the product.
ProductLine	nchar(2)		R = Road, M = Mountain, T = Touring, S = Standard
Class	nchar(2)		H = High, M = Medium, L = Low
Style	nchar(2)		W = Womens, M = Mens, U = Universal
ProductSubcategoryID	int	FK	Product is a member of this product subcategory. Foreign key to **ProductSubCategory.ProductSubCategoryID**.

ProductModelID	int	FK	Product is a member of this product model. Foreign key to **ProductModel.ProductModelID**.
SellStartDate	datetime	Not null	Date the product was available for sale.
SellEndDate	datetime		Date the product was no longer available for sale.
DiscontinuedDate	datetime		Date the product was discontinued.
rowguid	uniqueidentifier	Not null	ROWGUIDCOL number uniquely identifying the record. Used to support a merge replication sample.
ModifiedDate	datetime	Not null	Date and time the record was last updated.

Table 10. Product Master: Universal Record for Material Classification

G/ Production.BillsofMaterials

Attributes	Data Type	Key Linkage/ Nullable	Description
BillOfMaterialsID	int	PK/ Not null	Primary key for **BillOfMaterials** records.
ProductAssemblyID	int	FK	Parent product identification number. Foreign key to **Product.ProductID**.
ComponentID	int	FK/ Not null	Component identification number. Foreign key to **Product.ProductID**.
StartDate	datetime	Not null	Date the component started being used in the assembly item.
EndDate	datetime		Date the component stopped being used in the assembly item.
UnitMeasureCode	nchar(3)	FK/ Not null	Standard code identifying the unit of measure for the quantity.
BOMLevel	smallint	Not null	Indicates the depth the component is from its parent (**AssemblyID**).

PerAssemblyQty	decimal(8,2)	Not null	Quantity of the component needed to create the assembly.
ModifiedDate	datetime	Not null	Date and time the record was last updated.

Table 11. Bill of Materials (BOM): Structural Data for Material Requirements

H/ Production.ProductInventory

Attributes	Data Type	Key Linkage/ Nullable	Description
ProductID	int	PK, FK / Not null	Product identification number. Foreign key to Product.ProductID.
LocationID	smallint	PK, FK / Not null	Inventory location identification number. Foreign key to Location.LocationID.
Shelf	nvarchar(10)	Not null	Storage compartment within an inventory location.
Bin	tinyint	Not null	Storage container on a shelf in an inventory location.
Quantity	smallint	Not null	Quantity of products in the inventory location.
rowguid	uniqueidentifier	Not null	ROWGUIDCOL number uniquely identifying the record. Used to support a merge replication sample.
ModifiedDate	datetime	Not null	Date and time the record was last updated.

Table 12. Inventory Data: Dynamic Stock Levels for Optimization

1.3. Problem Statement

The current reliance on fragmented purchasing data and manual oversight (as detailed in Section 1) prevents CompanyX from proactively managing its material flow. This reactive approach has resulted in significant inventory imbalances—overstocking non-critical items while incurring stockouts on essential components like bearings and crankarms.

The primary challenge facing CompanyX's procurement function is the lack of codified, data-driven replenishment logic. Specifically, management cannot consistently answer two critical questions:

1. **When to Order (Timing):** At what precise inventory level (or forecasted production requirement) should a Material Purchase be triggered to prevent stockouts without inducing overstocking?
2. **Who to Order From (Vendor Selection):** Based on current supplier performance, pricing agreements, and material necessity, which vendor should fulfill the Purchase Order to optimize cost efficiency while ensuring On-Time, In-Full (OTIF) delivery?

This decision-making process currently lacks a unified analytical framework, resulting in inconsistent lead times and suboptimal unit pricing.

Project Goal: The objective of this initiative is to overcome these limitations by designing an analytical model that leverages the integrated ERP data structures. This model will automate material replenishment planning and provide data-driven vendor recommendations based on optimized cost, lead time, and performance metrics, ultimately ensuring uninterrupted material availability and significantly improving overall procurement efficiency.

2. Data Warehouse

2.1. Model Design

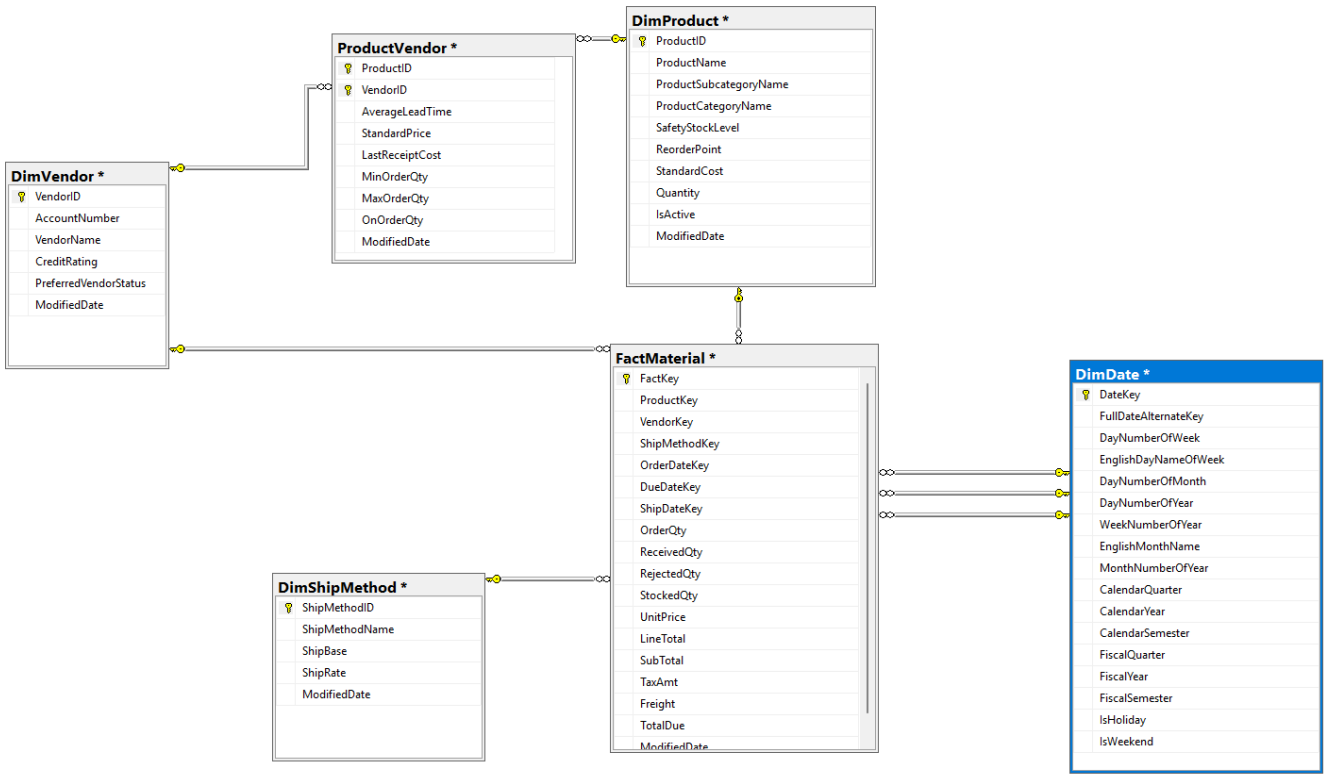


Figure 2. Data Warehouse Design

2.1.1. Overview

The proposed analytical structure for CompanyX's Material Purchase and Planning is designed using a Dimensional Modeling approach, specifically aligning with the Star Schema pattern. This design choice is optimal for supporting rapid querying, flexible reporting, and time-series analysis required for planning optimization.

The schema consists of one central Fact Table and multiple surrounding Dimension Tables, linked by foreign keys:

- The FactMaterial table serves as the core of the model, capturing all quantifiable events and metrics (e.g., ordered quantity, received quantity, unit price, and total due). This fact table is rich with foreign keys (ProductKey, VendorKey, ShipMethodKey, DateKey) allowing it to connect directly to the surrounding context dimensions.
- The Dimension Tables provide the necessary descriptive context for slicing and dicing the transactional data:
    - **DimProduct:** describes what material was purchased or tracked (category, standard cost).

- **DimVendor:** details who supplied the material (name, credit rating).
- **DimShipMethod:** provides context on how the material was delivered (carrier, rate).
- **DimDate:** offers a comprehensive time hierarchy (year, quarter, month, holiday flags) essential for accurate time-based forecasting and trend analysis.

Furthermore, the inclusion of **Bridge/Junction Tables**—specifically ProductVendor (linking Products to Vendors with specific pricing/lead times) and the BillOfMaterials (implied via FKs/relationship to the Product Dimension)—allows the model to accurately derive KPIs such as negotiated pricing and component consumption rates. This integrated structure moves CompanyX beyond simple transaction listing toward actionable insights on cost efficiency and material readiness.

2.1.2. Schema Structure

A. FACT TABLE:

The following breakdown dissects its key measures and foreign keys, illustrating exactly how transactional purchasing data is quantified and linked to contextual dimensions.

Attributes Name	Description
FactKey	Primary key uniquely identifying each fact record.
ProductKey	Foreign key linking to DimProduct, identifies the product involved in the transaction.
VendorKey	Foreign key linking to DimVendor, identifies the vendor supplying the material.
ShipMethodKey	Foreign key linking to DimShipMethod, indicates the shipping method used.
OrderDateKey	Foreign key linking to DimDate, representing the date when the order was placed.
DueDateKey	Foreign key linking to DimDate, representing the expected delivery date.
ShipDateKey	Foreign key linking to DimDate, representing the actual shipping date.
OrderQty	Quantity of materials ordered.

ReceivedQty	Quantity of materials actually received.
RejectedQty	Quantity of materials rejected due to quality or other issues.
StockedQty	Quantity of materials accepted into inventory.
UnitPrice	Price per unit of the material.
LineTotal	Total cost for the line item (OrderQty × UnitPrice).
SubTotal	Total before tax and freight.
TaxAmt	Tax amount applied to the order.
Freight	Freight cost of shipping the materials.
TotalDue	Total amount due including tax and freight.
ModifiedDate	Timestamp of the last update to the record.

Table 13. FactMaterial Breakdown

B. DIMENSION TABLE:

a/ DimProduct

The DimProduct table serves as the universal master catalog, containing descriptive attributes for every material or product (both purchased raw materials and finished components). Key attributes include ProductName, ProductCategoryName, and vital planning parameters such as SafetyStockLevel and ReorderPoint.

Attributes Name	Description
ProductID	Primary key uniquely identifying a product.
ProductName	Name of the product.

ProductSubcategoryName	Subcategory to which the product belongs.
ProductCategoryName	High-level category of the product.
SafetyStockLevel	Minimum quantity of stock to maintain for safety.
ReorderPoint	Inventory level at which a new order should be placed.
StandardCost	Standard manufacturing or purchasing cost.
Quantity	Current quantity in stock.
iActive	Indicates whether the product is active (1) or inactive (0).
ModifiedDate	Timestamp of the last modification.

Table 14: DimProduct Breakdown

b/ DimVendor

The DimVendor table contains essential descriptive information such as VendorName, AccountNumber, and crucial performance metrics like CreditRating and PreferredVendorStatus. This dimension is fundamental to the analysis, enabling the evaluation of supplier performance and reliability, which is essential for making data-driven vendor selection recommendations.

Attributes Name	Description
VendorID	Primary key uniquely identifying a vendor.
AccountNumber	Vendor's account number in the system.
VendorName	Name of the vendor or supplier.
CreditRating	Credit rating of the vendor for risk assessment.

PreferredVendorStatus	Indicates if the vendor is marked as preferred.
ModifiedDate	Timestamp of the last modification.

Table 15: DimVendor Breakdown

c/ DimShipMethod

The DimShipMethod table contains descriptive information regarding the various logistics and delivery carriers used by CompanyX. Key attributes like ShipMethodName, ShipBase, and ShipRate are captured here. This dimension is crucial for calculating the total landed cost of materials and supporting the analysis of delivery performance and cost efficiency across different shipping methods.

Attributes Name	Description
ShipMethodID	Primary key uniquely identifying a shipping method.
ShipMethodName	Name of the shipping method (e.g., Air, Ground, Sea).
ShipBase	Base cost associated with using this shipping method.
ShipRate	Rate charged per unit of distance or weight.
ModifiedDate	Timestamp of the last modification.

Table 16: DimShipmethod Breakdown

d/ DimDate

The DimDate table is a standard Date Dimension, providing a comprehensive set of time-based attributes essential for flexible time-series analysis. Key attributes include the primary key FullDateAlternateKey and granular period breakdowns such as CalendarYear, FiscalQuarter, and boolean flags like IsHoliday and IsWeekend. This dimension allows analysts to easily slice purchasing and inventory metrics across calendar periods, fiscal periods, and specific temporal contexts.

Attributes Name	Description
DateKey	Surrogate key representing each date (often in YYYYMMDD format).
FullDateAlternateKey	Actual calendar date value.
DayNumberOfWeek	Numeric representation of the day in the week (1 = Sunday, 7 = Saturday).
EnglishDayNameOfWeek	Name of the day (e.g., Monday, Tuesday).
DayNumberOfMonth	Day number within the month.
DayNumberOfYear	Day number within the year.
WeekNumberOfYear	Week number within the year.
EnglishMonthName	Full month name (e.g., January).
MonthNumberOfYear	Numeric month value (1–12).
CalendarQuarter	Quarter of the year (1–4).
CalendarYear	Calendar year (e.g., 2025).
CalendarSemester	Half-year period (1 or 2).
FiscalQuarter	Quarter in the fiscal year.
FiscalYear	Fiscal year.
FiscalSemester	Half-year period in the fiscal year.

IsHoliday	Indicates whether the date is a holiday (1 = Yes, 0 = No).
IsWeekend	Indicates whether the date falls on a weekend.

Table 17: DimDate Breakdown

e/ ProductVendor

The ProductVendor table functions as a critical Bridge/Relationship Table, explicitly linking specific materials (ProductID) to the vendors who supply them (VendorID). It details vendor-specific commercial terms, capturing vital planning attributes such as AverageLeadTime, negotiated StandardPrice, and order constraints (MinOrderQty, MaxOrderQty). This structure is essential for establishing baseline costs and supplier reliability benchmarks per product line, directly supporting strategic vendor selection during the replenishment recommendation phase.

Attributes Name	Description
ProductID	Foreign key linking to DimProduct.
VendorID	Foreign key linking to DimVendor.
AverageLeadTime	Average number of days taken by the vendor to deliver the product.
StandardPrice	Standard unit price set by the vendor.
LastReceiptCost	Cost at which the last batch was received.
MinOrderQty	Minimum quantity that can be ordered from the vendor.
MaxOrderQty	Maximum quantity allowed per order.
OnOrderQty	Quantity currently on order but not yet received.

Table 18. ProductVendor Breakdown

2.2. ETL Pipeline Implementation

2.2.1. ETL Overview

The ETL (Extract, Transform, Load) pipeline serves as the critical data integration backbone for CompanyX's Material Purchase and Planning data warehouse, orchestrating the systematic movement of data from operational ERP systems into the analytical star schema. This comprehensive pipeline is implemented using SQL Server Integration Services (SSIS) and is designed to support both initial data loading and ongoing incremental updates with near real-time capabilities.

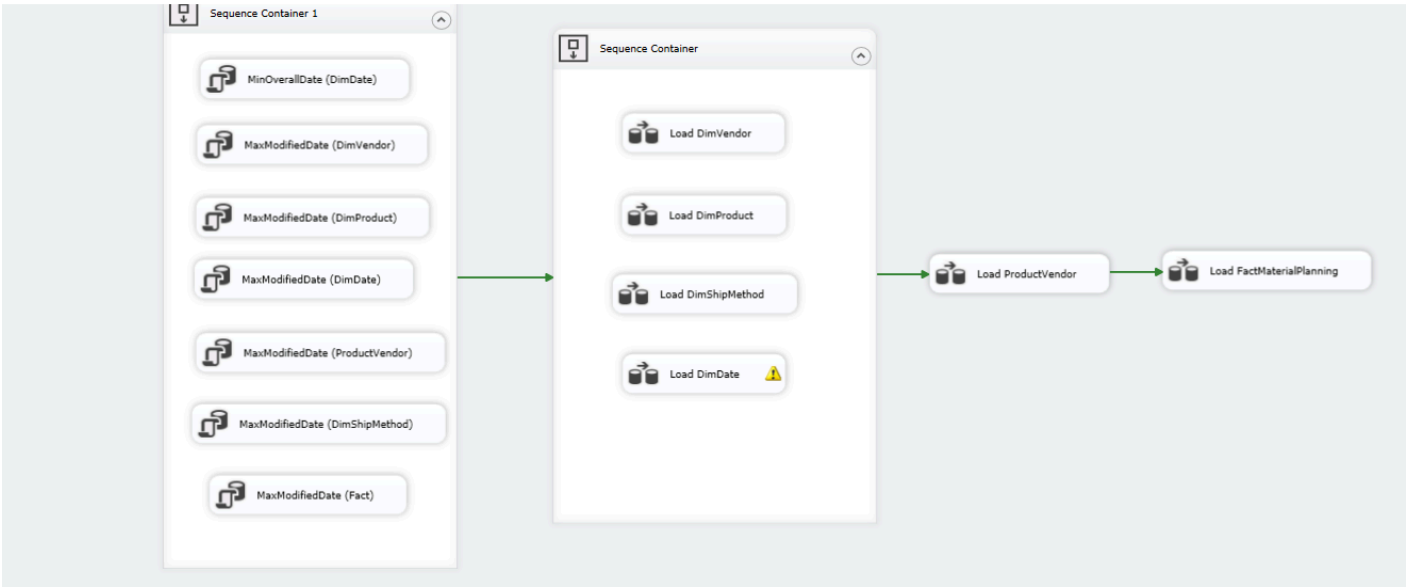


Image: Control Flow of ETL Pineline

2.2.1.1. DimProduct

```
SELECT
  p.ProductID,
  p.Name AS ProductName,
  ps.Name AS ProductSubcategoryName,
  pc.Name AS ProductCategoryName,
  p.SafetyStockLevel,
  p.ReorderPoint,
  p.StandardCost,
  ISNULL(SUM(pi.Quantity), 0) AS Quantity,
  CASE
    WHEN p.SellEndDate IS NULL OR p.SellEndDate > GETDATE() THEN 1
    ELSE 0
  END AS IsActive,
```

```
  MAX(COALESCE(p.ModifiedDate, ps.ModifiedDate, pc.ModifiedDate, '1900-01-01')) AS ModifiedDate
FROM Production.Product AS p
LEFT JOIN Production.ProductSubcategory AS ps ON p.ProductSubcategoryID = ps.ProductSubcategoryID
LEFT JOIN Production.ProductCategory AS pc ON ps.ProductCategoryID = pc.ProductCategoryID
LEFT JOIN Production.ProductInventory AS pi ON p.ProductID = pi.ProductID
GROUP BY p.ProductID, p.Name, ps.Name, pc.Name, p.SafetyStockLevel, p.ReorderPoint, p.StandardCost, p.SellEndDate
```

Extracted via complex SQL query joining multiple Production schema tables:

- **Primary Sources:** Production.Product (master), Production.ProductSubcategory, Production.ProductCategory, Production.ProductInventory.

- **Extraction Method:** Multi-table JOIN with aggregation to calculate current inventory levels.

- **Key Attributes:** ProductID (PK), ProductName, SafetyStockLevel, ReorderPoint, StandardCost, aggregated Quantity, derived IsActive flag.

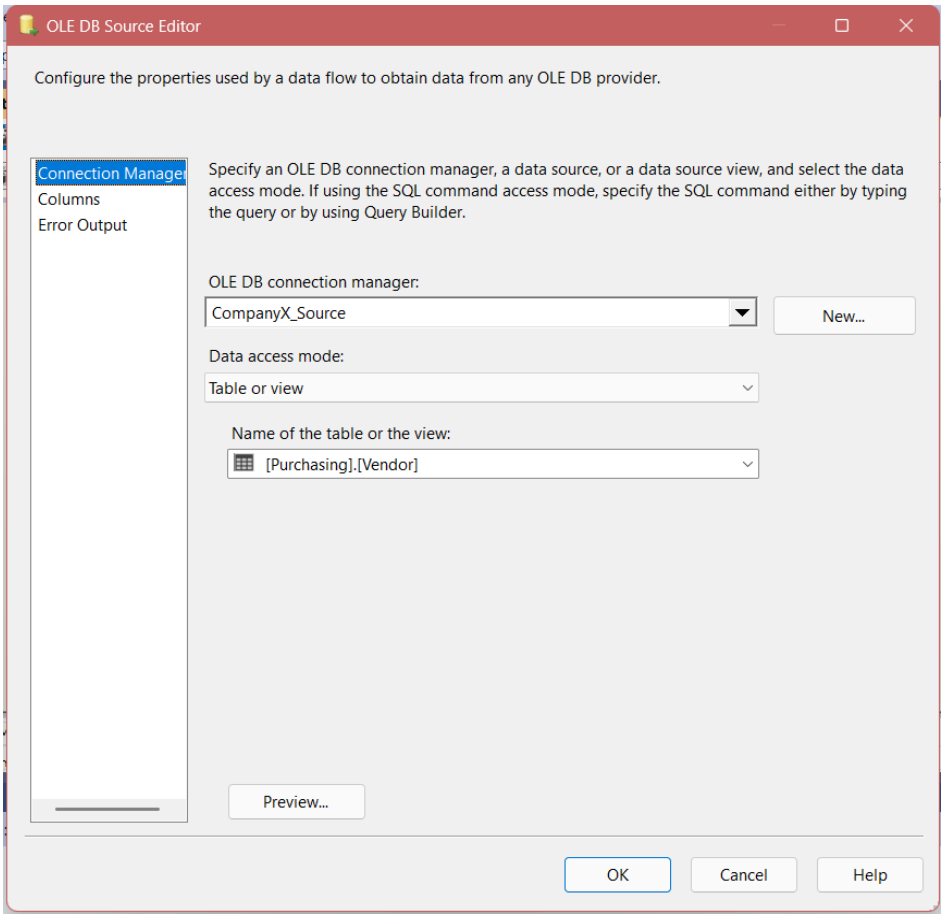
2.2.1.2. DimVendor

Direct extraction from single Purchasing schema table:

- **Primary Source:** Purchasing.Vendor.

- **Extraction Method:** Simple SELECT with direct column mapping.

- **Key Attributes:** BusinessEntityID (PK → VendorKey), AccountNumber, VendorName, CreditRating, PreferredVendorStatus, ActiveFlag.


OLE DB Source Editor
Configure the properties used by a data flow to obtain data from any OLE DB provider.
Connection Manager
Columns
Error Output
Specify an OLE DB connection manager, a data source, or a data source view, and select the data access mode. If using the SQL command access mode, specify the SQL command either by typing the query or by using Query Builder.
OLE DB connection manager:
CompanyX_Source
New...
Data access mode:
Table or view
Name of the table or the view:
[Purchasing].[Vendor]
Preview...
OK
Cancel
Help

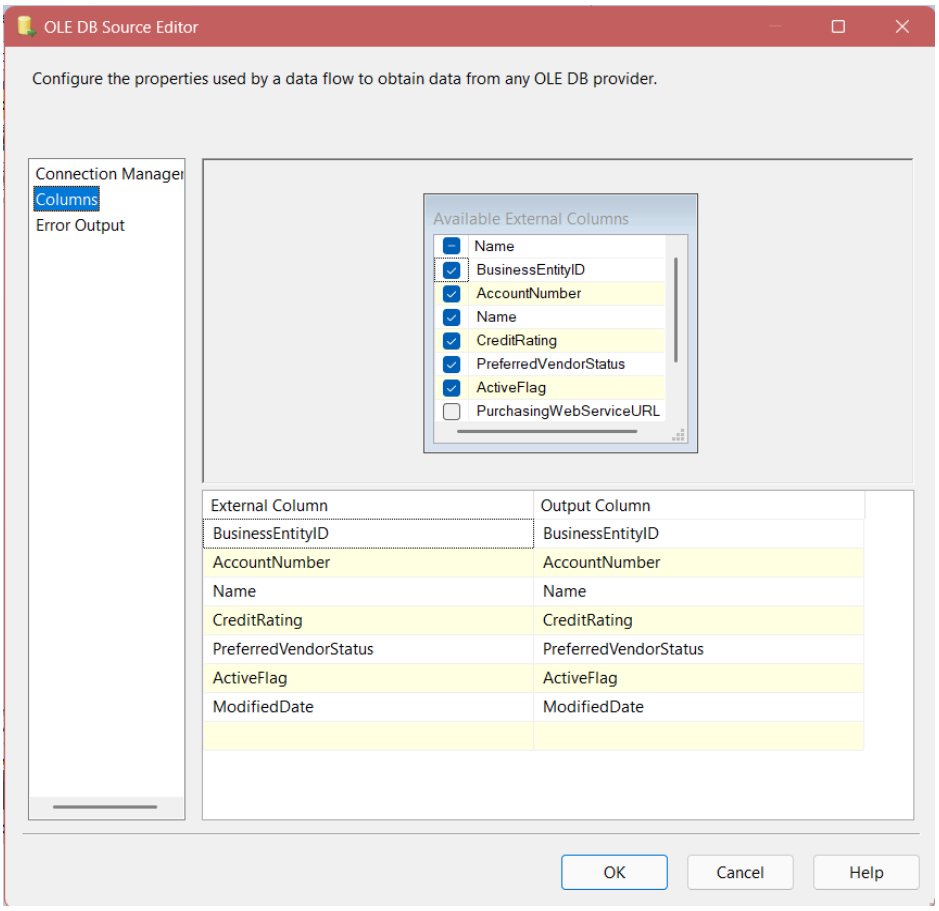


Figure 3: Choose the source column

2.2.1.3. DimShipmethod

```
SELECT
   ShipMethodID,
   Name AS ShipMethodName,
   ShipBase,
   ShipRate,
   ModifiedDate
FROM Purchasing.ShipMethod;
```

Straightforward extraction from logistics reference table:

- **Primary Source:** Purchasing.ShipMethod.

- **Extraction Method:** Direct SELECT with column aliasing.

- **Key Attributes:** ShipMethodID (PK), ShipMethodName, ShipBase, ShipRate for cost analysis.

2.2.1.4. DimDate

```
public override void CreateNewOutputRows()
```

```
{
   /*
    Add rows by calling the AddRow method on the member variable named "<Output Name>Buffer".
    For example, call MyOutputBuffer.AddRow() if your output was named "MyOutput".
   */
   DateTime minDate = Variables.OverallMinDate;
   DateTime maxDate = Variables.OverallMaxDate;
   DateTime currentDate = minDate;

   while (currentDate <= maxDate)
   {
      Output0Buffer.AddRow();
      Output0Buffer.DateKey = int.Parse(currentDate.ToString("yyyyMMdd"));
      Output0Buffer.FullDateAlternateKey = currentDate;
      Output0Buffer.DayNumberOfWeek = (byte)((byte)currentDate.DayOfWeek + 1);  // 1=Sunday
      Output0Buffer.EnglishDayNameOfWeek = currentDate.DayOfWeek.ToString();
      Output0Buffer.DayNumberOfMonth = (byte)currentDate.Day;
      Output0Buffer.DayNumberOfYear = (short)currentDate.DayOfYear;
      Output0Buffer.WeekNumberOfYear = (byte)GetIsoWeek(currentDate);  // Custom ISO week func
      Output0Buffer.EnglishMonthName = currentDate.ToString("MMMM");
      Output0Buffer.MonthNumberOfYear = (byte)currentDate.Month;
      Output0Buffer.CalendarQuarter = (byte)((currentDate.Month - 1) / 3 + 1);
      Output0Buffer.CalendarYear = (short)currentDate.Year;
      Output0Buffer.CalendarSemester = (byte)((currentDate.Month <= 6) ? 1 : 2);

      // Fiscal (July-June example)
      DateTime fiscalAdj = currentDate.AddMonths(-6);
      byte fiscalQ = (byte)((fiscalAdj.Month - 1) / 3 + 1);
      Output0Buffer.FiscalQuarter = fiscalQ;
            Output0Buffer.FiscalYear = (short)((currentDate.Month >= 7) ? currentDate.Year + 1 :
currentDate.Year);
      Output0Buffer.FiscalSemester = (byte)((fiscalQ <= 2) ? 1 : 2);

      Output0Buffer.IsHoliday = false;  // Set manually later or add logic
                  Output0Buffer.IsWeekend = (currentDate.DayOfWeek == DayOfWeek.Saturday ||
currentDate.DayOfWeek == DayOfWeek.Sunday);

      currentDate = currentDate.AddDays(1);
   }
}
// Helper for ISO Week (since .NET doesn't have direct equiv)
private int GetIsoWeek(DateTime date)
{
   int dayOfYear = date.DayOfYear;
   int week = (dayOfYear - 1) / 7 + 1;
```

```
   DayOfWeek day = date.DayOfWeek;
    if (week == 53 && day != DayOfWeek.Thursday && (date.Year % 4 == 0 && date.Year % 100 != 0 ||
date.Year % 400 == 0 && day != DayOfWeek.Wednesday))
      week = 1;
   return week;
}
```

Unique generation-based approach (not extracted from source):

- **Primary Source:** Generated in-memory using SSIS Script Component.

- **Extraction Method:** Programmatic date range generation from configurable start/end variables.

- **Key Attributes:** DateKey (YYYYMMDD), FullDateAlternateKey, comprehensive calendar and fiscal hierarchies, IsHoliday, IsWeekend flags.

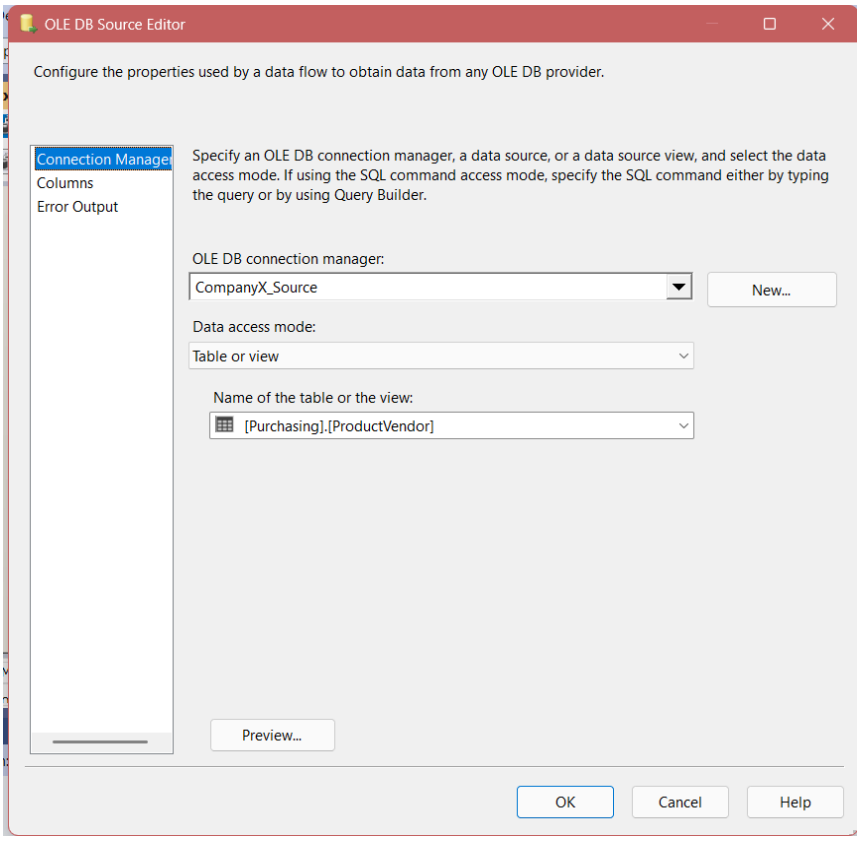
2.2.1.5. ProductVendor

Critical vendor-product relationship extraction:

- **Primary Source:** Purchasing.ProductVendor.

- **Extraction Method:** Direct mapping with foreign key relationships to both Product and Vendor dimensions.

- **Key Attributes:** ProductID (PK), BusinessEntityID/VendorID (PK), AverageLeadTime, StandardPrice, LastReceiptCost, MinOrderQty, MaxOrderQty, OnOrderQty.


OLE DB Source Editor
Configure the properties used by a data flow to obtain data from any OLE DB provider.
Connection Manager
Columns
Error Output
Specify an OLE DB connection manager, a data source, or a data source view, and select the data access mode. If using the SQL command access mode, specify the SQL command either by typing the query or by using Query Builder.
OLE DB connection manager:
CompanyX_Source
New...
Data access mode:
Table or view
Name of the table or the view:
[Purchasing].[ProductVendor]
Preview...
OK
Cancel
Help

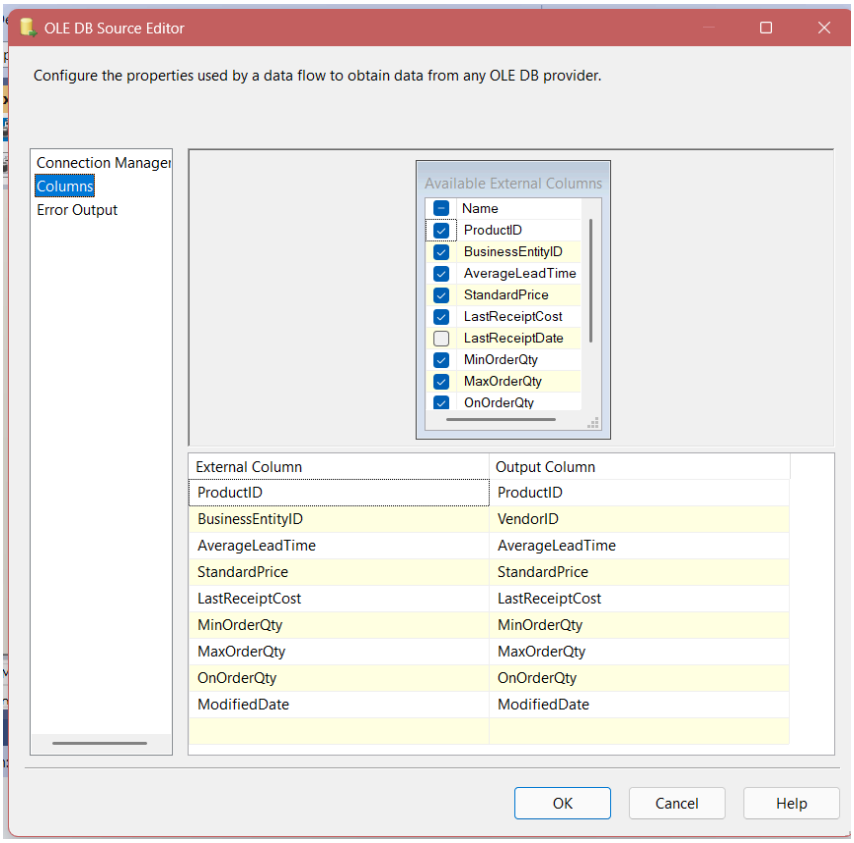

OLE DB Source Editor
Configure the properties used by a data flow to obtain data from any OLE DB provider.
Connection Manager
Columns
Error Output
Available External Columns
Name
ProductID
BusinessEntityID
AverageLeadTime
StandardPrice
LastReceiptCost
LastReceiptDate
MinOrderQty
MaxOrderQty
OnOrderQty
External Column
Output Column
ProductID
ProductID
BusinessEntityID
VendorID
AverageLeadTime
AverageLeadTime
StandardPrice
StandardPrice
LastReceiptCost
LastReceiptCost
MinOrderQty
MinOrderQty
MaxOrderQty
MaxOrderQty
OnOrderQty
OnOrderQty
ModifiedDate
ModifiedDate
OK
Cancel
Help

Figure 4: Choose the source column

2.2.1.6. FactMaterial

```
SELECT
   pod.ProductID,
   poh.VendorID,
   poh.ShipMethodID,
   poh.OrderDate,
   pod.DueDate,
   poh.ShipDate,
   pod.OrderQty,
   pod.ReceivedQty,
   pod.RejectedQty,
   pod.StockedQty,
   pod.UnitPrice,
   pod.LineTotal,
   poh.SubTotal,
   poh.TaxAmt,
   poh.Freight,
   poh.TotalDue,
   MAX(COALESCE(poh.ModifiedDate, pod.ModifiedDate, '1900-01-01')) AS ModifiedDate
FROM Purchasing.PurchaseOrderHeader AS poh
INNER JOIN Purchasing.PurchaseOrderDetail AS pod
   ON poh.PurchaseOrderID = pod.PurchaseOrderID
INNER JOIN Purchasing.Vendor AS v
   ON poh.VendorID = v.BusinessEntityID
GROUP BY
   pod.ProductID,
   poh.VendorID,
   poh.ShipMethodID,
   poh.OrderDate,
   pod.DueDate,
   poh.ShipDate,
   pod.OrderQty,
   pod.ReceivedQty,
   pod.RejectedQty,
   pod.StockedQty,
   pod.UnitPrice,
   pod.LineTotal,
   poh.SubTotal,
   poh.TaxAmt,
   poh.Freight,
   poh.TotalDue
```

Complex multi-table extraction capturing complete purchase transaction lifecycle:

- **Primary Sources:** Purchasing.PurchaseOrderHeader ⟷ Purchasing.PurchaseOrderDetail ⟷ Purchasing.Vendor.

- **Extraction Method:** Multi-table INNER JOIN with GROUP BY aggregation and change tracking

- **Key Measures:** OrderQty, ReceivedQty, RejectedQty, StockedQty, UnitPrice, LineTotal, SubTotal, TaxAmt, Freight, TotalDue.

- **Foreign Key Resolution:** ProductID, VendorID, ShipMethodID, multiple date dimensions (OrderDate, DueDate, ShipDate).

The ETL pipeline implements a sophisticated surrogate key management system to optimize analytical performance and maintain referential integrity:

- **DimProduct:** ProductID (natural key) → ProductKey (surrogate integer key)
- **DimVendor:** BusinessEntityID (natural key) → VendorKey (surrogate integer key)
- **DimShipMethod:** ShipMethodID (natural key) → ShipMethodKey (surrogate integer key)
- **DimDate:** Computed DateKey in YYYYMMDD format (e.g., 20251027)

The fact table ETL process performs multiple lookup transformations to resolve natural keys to surrogate keys:

- **ProductKey:** Lookup ProductID in DimProduct to obtain ProductKey.

- **VendorKey:** Lookup VendorID in DimVendor to obtain VendorKey.

- **ShipMethodKey:** Lookup ShipMethodID in DimShipMethod to obtain ShipMethodKey.

- **Date Keys:** Convert OrderDate, DueDate, ShipDate to corresponding DateKeys through DimDate lookups.

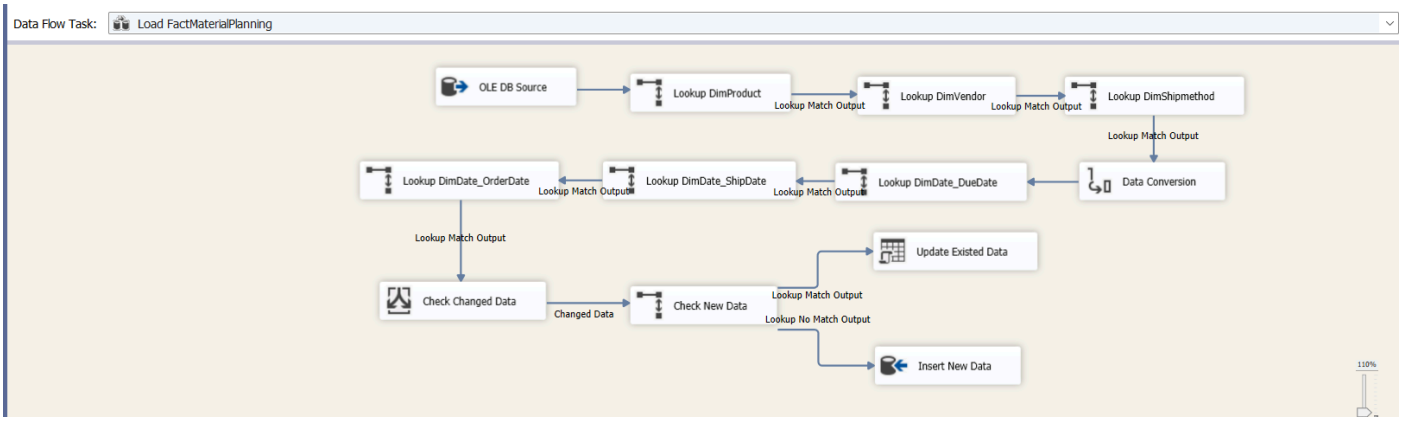


Figure 5: FactMaterial Data Flow

2.2.2. Data Transformation

The **Transformation phase** is the core of the ETL pipeline, where raw, heterogeneous data extracted from CompanyX's is **cleaned, standardized, enriched, and reshaped** to conform to the **star schema** of the data warehouse. This phase ensures data quality, consistency, and analytical readiness.

All transformations are implemented using **SQL Server Integration Services (SSIS)** with a combination of **Data Flow Tasks**, **Derived Column Transformations**, **Lookup Transformations**, **Conditional Splits**, **Aggregate Components**, and **Slowly Changing Dimension (SCD) Wizards**. The transformation logic is designed to support **incremental loading** and **near real-time updates** via change tracking and efficient key lookups.

2.2.2.1. Key Transformation Principles

Principle	Implementation
Data Cleansing	Remove NULLs, standardize formats (e.g., dates, currency), handle missing values.
Surrogate Key Generation	Replace natural keys (e.g., ProductID) with integer surrogate keys (ProductKey).
Dimensional Conformance	Align source data with dimension and fact table structures.
Historical Tracking	Use **SCD Type 1** (overwrite) for changes and **SCD Type 2** (new row).
Performance Optimization	Use in-memory lookups, cached connection managers, and fast-load options.

Table 19: Key Transformation Principles

2.2.2.2. Dimension Table Transformations

DimProduct

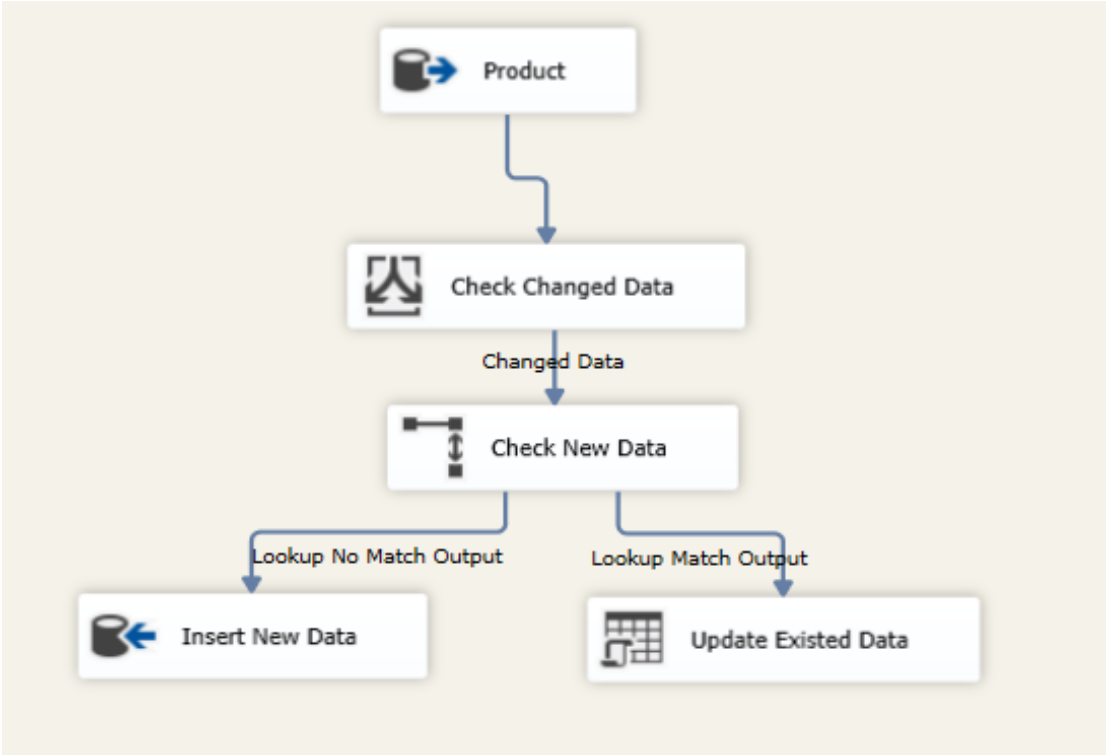


Figure 6: DimProduct Dataflow

- **Source**: Production.Product, ProductSubcategory, ProductCategory, ProductInventory
- **Key Transformation Steps**:
    1. **Conditional Split:** Detect changed records using the condition (DT_DATE)ModifiedDate > (DT_DATE)@[User::LastLoadDate_Product] || ISNULL(@[User::LastLoadDate_Product]). This routes qualifying records to the "Changed Data" output for subsequent handling.
    2. **Lookup Transformation:** Match on ProductID to check against existing DimProduct records. If no match is found (Lookup No Match Output), direct the flow to insert new data; if a match exists (Lookup Match Output), route to update the corresponding records.
    3. **Insert/Update Destinations:** Insert new rows into the DimProduct table for unmatched data, while updating existing rows for matched, changed records to ensure ongoing data accuracy and integrity.

DimVendor

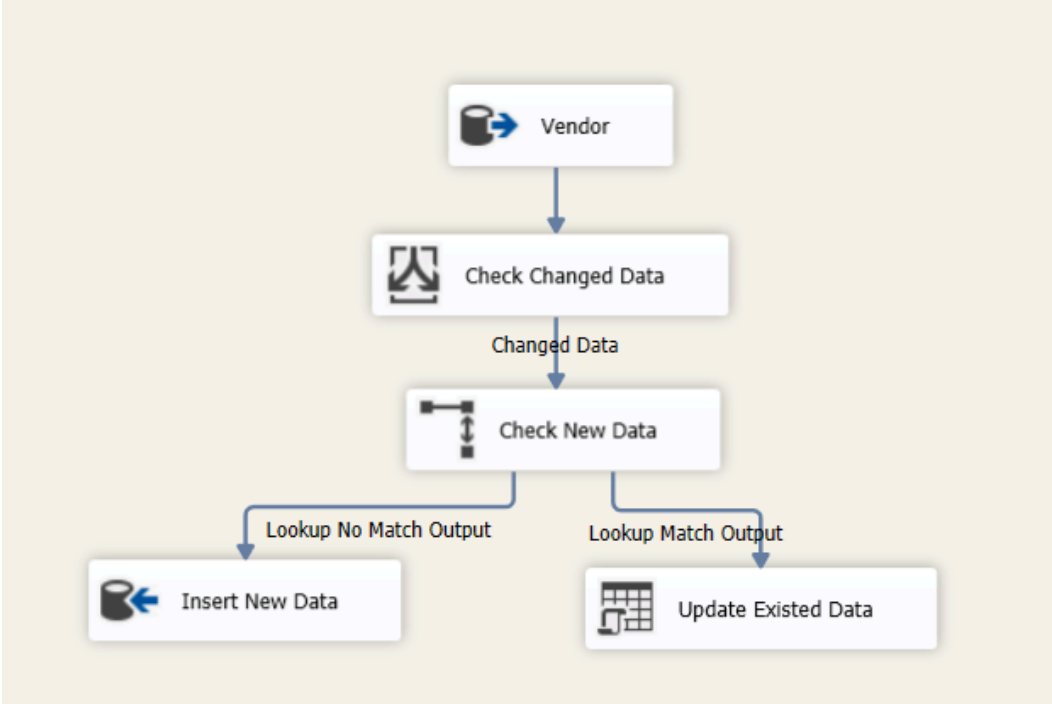


Figure 7: DimVendor Dataflow

- **Source**: Purchasing.Vendor
- **Key Transformation Steps**:
    1. **Conditional Split:** Detect changed records using the condition (DT_DATE)ModifiedDate > (DT_DATE)@[User::LastLoadDate_Vendor] || ISNULL(@[User::LastLoadDate_Vendor]). This routes qualifying records to the "Changed Data" output for subsequent handling.
    2. **Lookup Transformation:**Match on BusinessEntityID to VendorID to check against existing DimVendor records. If no match is found (Lookup No Match Output), direct the flow to insert new data; if a match exists (Lookup Match Output), route to update the corresponding records.
    3. **Insert/Update Destinations:** Insert new rows into the DimVendor table for unmatched data, while updating existing rows for matched, changed records to ensure ongoing data accuracy and integrity.

DimShipMethod

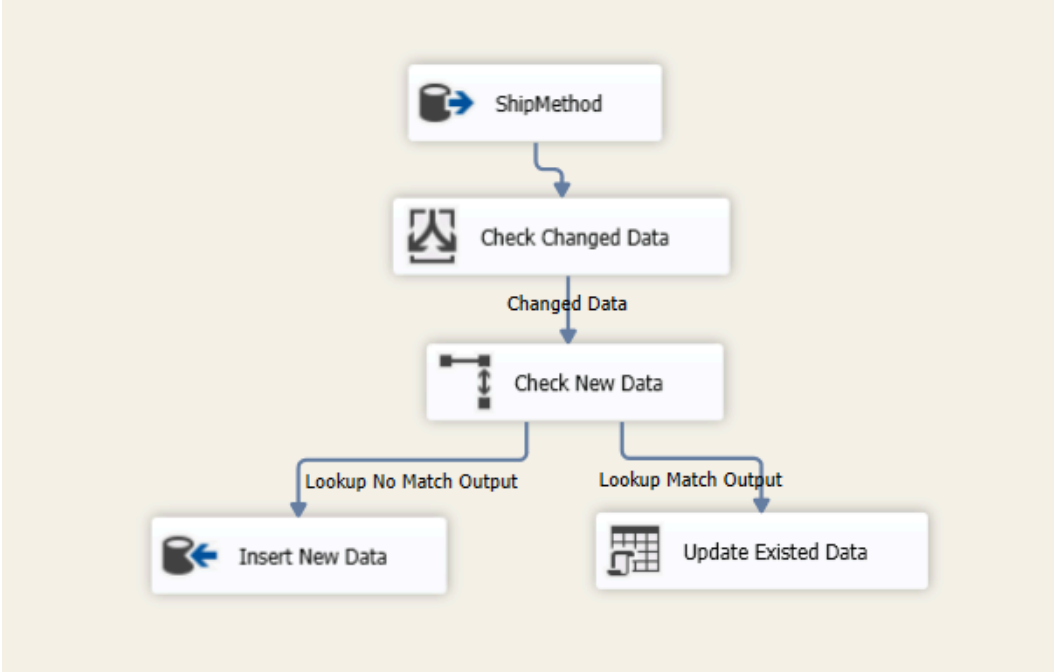


Figure 8: DimShipMethod Dataflow

- **Source**: Purchasing.ShipMethod
- **Key Transformation Steps**:
    1. **Conditional Split:** Detect changed records using the condition (DT_DATE)ModifiedDate > (DT_DATE)@[User::LastLoadDate_ShipMethod] || ISNULL(@[User::LastLoadDate_ShipMethod]). This routes qualifying records to the "Changed Data" output for subsequent handling.
    2. **Lookup Transformation:** Match on ShipMethodID to check against existing DimShipMethod records. If no match is found (Lookup No Match Output), direct the flow to insert new data; if a match exists (Lookup Match Output), route to update the corresponding records.
    3. **Insert/Update Destinations:** Insert new rows into the DimShipMethod table for unmatched data, while updating existing rows for matched, changed records to ensure ongoing data accuracy and integrity.

DimDate

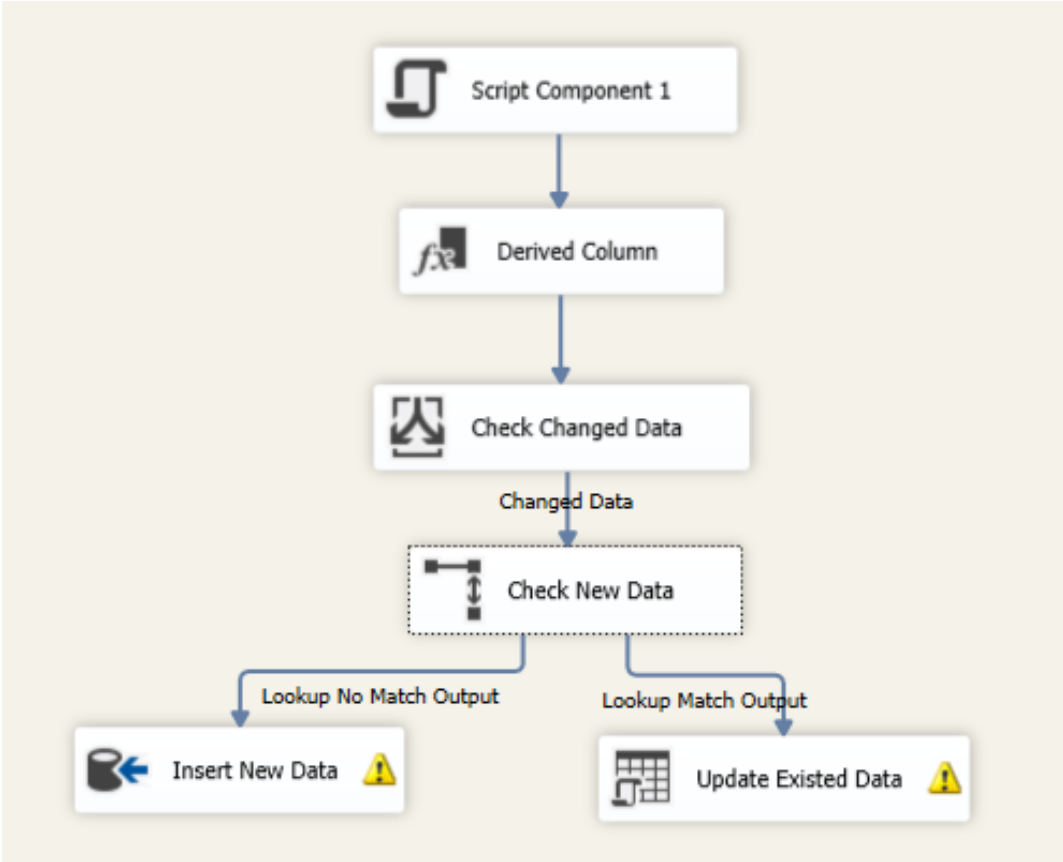


Figure 9: DimDate Dataflow

- **Source**: Generated programmatically via Script Component (no direct database source; dates are created dynamically based on calculated range). Generate date rows from a predefined range (e.g., MinDate to MaxDate stored in variables), computing calendar and fiscal attributes such as DateKey, DayNumberOfWeek, FiscalYear, etc., using C# logic.
- **Key Transformation Steps**:
    1. **Derived Column:** Add ModifiedDate as a new column with the expression (DT_DBDATE)GETDATE() to timestamp the records for auditing and change tracking.
    2. **Conditional Split:** Detect changed records using the condition (DT_DATE)ModifiedDate > (DT_DATE)@[User::LastLoadDate_Date] || ISNULL(@[User::LastLoadDate_Date]). This routes qualifying records to the "Changed Data" output for subsequent handling.
    3. **Lookup Transformation:** Match on DateKey to check against existing DimDate records. If no match is found (Lookup No Match Output), direct the flow to insert new data; if a match exists (Lookup Match Output), route to update the corresponding records.

4. **Insert/Update Destinations:** Insert new rows into the DimDate table for unmatched data, while updating existing rows for matched, changed records to ensure ongoing data accuracy and integrity.

ProductVendor (Bridge Table)

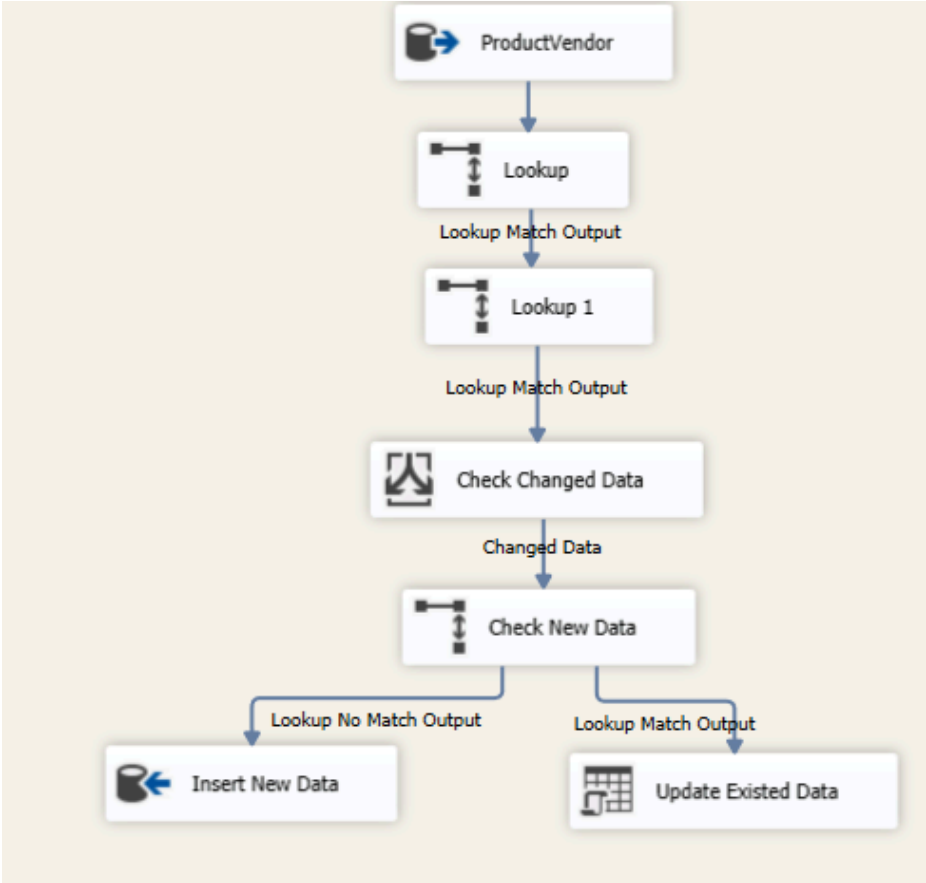


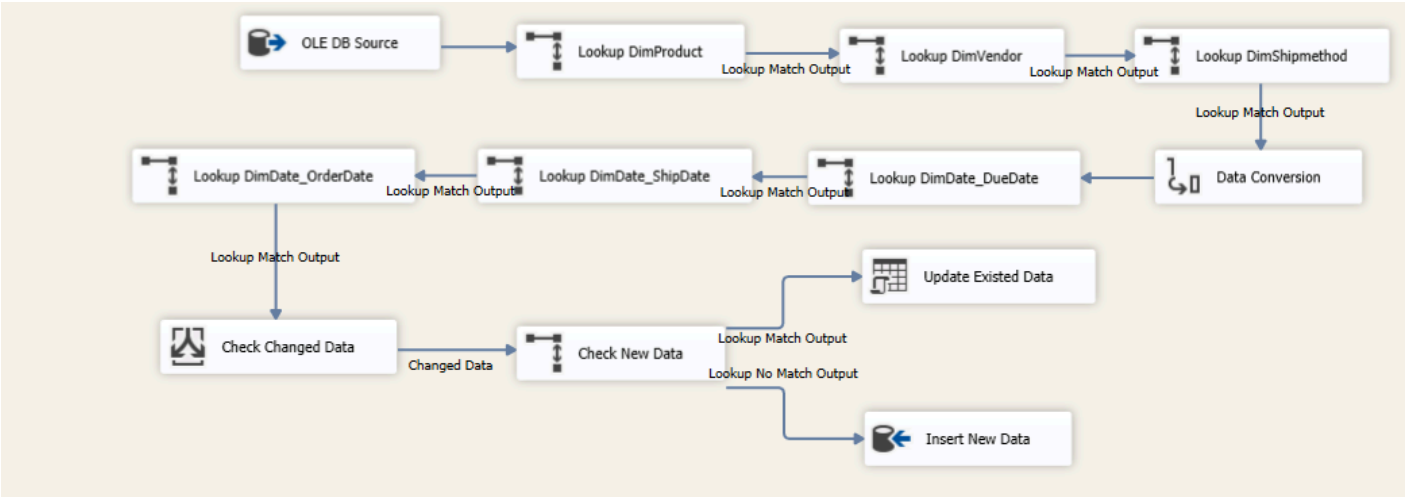
Figure 10: ProductVendor (Bridge Table) Dataflow

- **Source**: Purchasing.ProductVendor
- **Key Transformation Steps**:
  1. **Lookup Transformation #1**: Map ProductID to ProductKey from DimProduct to resolve the product reference.
  2. **Lookup Transformation #2**: Map BusinessEntityID to VendorKey from DimVendor to resolve the vendor reference.
  3. **Conditional Split:** Detect changed records using the condition (DT_DATE)ModifiedDate > (DT_DATE)@[User::LastLoadDate_ProductVendor] || ISNULL(@[User::LastLoadDate_ProductVendor]). This routes qualifying records to the "Changed Data" output for subsequent handling.
  4. **Lookup Transformation #3:** Perform a match on the composite keys (e.g., ProductKey and VendorKey) against existing records in ProductVendor. If no match is found (Lookup No Match

Output), direct the flow to insert new data; if a match exists (Lookup Match Output), route to update the corresponding records.

5. **Insert/Update Destinations:** Insert new rows into the ProductVendor table for unmatched data, while updating existing rows for matched, changed records to ensure ongoing data accuracy and integrity.

2.2.2.3. Fact Table Transformation: FactMaterialPlanning



- **Source**: PurchaseOrderHeader, PurchaseOrderDetail, Vendor
- **Key Transformation Steps**:
  1. **Multiple Lookup Transformations (four in sequence)**:
     - Map ProductID to ProductKey from DimProduct.
     - Map VendorID to VendorKey from DimVendor.
     - Map ShipMethodID to ShipMethodKey from DimShipMethod.
     - Map OrderDate, DueDate, and ShipDate to their respective surrogate keys (OrderDateKey, DueDateKey, ShipDateKey) from DimDate.
  2. **Conditional Split:** Detect changed records using the condition (DT_DATE)ModifiedDate > (DT_DATE)@[User::LastLoadDate_Fact] || ISNULL(@[User::LastLoadDate_Fact]). This routes qualifying records to the "Changed Data" output for subsequent handling.
  3. **Lookup Transformation :** Perform a composite match on all keys (e.g., ProductKey, VendorKey, etc.) against existing records in FactMaterialPlanning. If no match is found (Lookup No Match Output), direct the flow to insert new data; if a match exists (Lookup Match Output), route to update the corresponding records.
  4. **Insert/Update Destinations:** Insert new rows into FactMaterialPlanning for unmatched data, while updating existing rows for matched, changed records to ensure ongoing data accuracy and integrity.

2.2.3. Data Loading

The **Data Loading phase** is the final step in the ETL pipeline, where transformed data is inserted or updated into the target data warehouse tables. This phase ensures efficient population of the star schema while minimizing downtime and resource usage. The loading process is implemented using **SQL Server Integration Services (SSIS)** packages, specifically Load_MaterialDW_.dtsx, which handles dimensions (DimProduct, DimVendor, DimShipMethod, DimDate), the bridge table (ProductVendor), and the fact table (FactMaterialPlanning).

Loading supports **incremental updates** to process only new or changed data, and is designed for **near real-time synchronization** through scheduled executions. A sample execution log from October 27, 2025, demonstrates the process, showing validation, caching, and successful completion with no new rows inserted (indicating no deltas since the last load).

Incremental Data Loading Process

Incremental loading processes only new or modified data since the last ETL run, using timestamps (ModifiedDate) and user variables (e.g., [User::LastLoadDate]) to detect deltas. This avoids reloading the entire dataset, reducing processing time and resource consumption.

- **Process Steps**:
  - **Validation Phase**: SSIS validates data flow components, connections, and column mappings. Warnings may occur if external columns are out of sync (e.g., due to schema changes), but these are resolvable by updating metadata.
  - **Prepare/Pre-Execute Phase**: Caches lookup data (e.g., 505 rows for DimProduct, 105 for DimVendor, 36,327 for DimDate) for efficient matching.
  - **Execute Phase**:
    - **Change Detection**: Conditional Splits and Lookups identify new/changed rows (e.g., ModifiedDate > LastLoadDate).
    - **Routing**: New rows → Insert; Changed rows → Update; Unchanged → Ignored.
    - **Caching & Performance**: Unique rows are cached (e.g., 8,839 for FactMaterialPlanning), with low processing times (e.g., 0.016 seconds).
  - **Post-Execute/Cleanup**: Commits inserts/updates and clears buffers.
- **Execution Log Insights** (from run on October 27, 2025):
  - No rows written to destinations (e.g., "Insert New Data" wrote 0 rows across all tables), indicating no deltas detected—confirming incremental efficiency.
  - Caching optimized lookups (e.g., 36,327 rows for date lookups processed in ~0.047 seconds).
  - Warnings (e.g., column sync issues in DimDate) highlight maintenance needs but did not halt execution.
  - Total run: Success, with no errors.

Near Real-time Updates

Near real-time updates are achieved by scheduling frequent executions of the SSIS package (Load_MaterialDW_.dtsx) using **SQL Server Agent Jobs**, ensuring the data warehouse reflects source changes with minimal latency (e.g., every 15–30 minutes). This approach balances performance and freshness, suitable for procurement analytics requiring timely insights without full real-time overhead.

- **Implementation Steps**:
    1. **Create SQL Agent Job**: In SQL Server Management Studio (SSMS), navigate to SQL Server Agent > Jobs > New Job. Name it (e.g., "MaterialDW_IncrementalLoad") and assign the SSIS package as a step (Type: SQL Server Integration Services Package; Package source: File system or SSISDB).
    2. **Schedule Configuration**: In the Job Properties > Schedules tab, create a new schedule:
        - Schedule Type: Recurring.
        - Frequency: Daily, with occurrences every 15 minutes (e.g., Starts: 00:00, Recurs every: 15 minutes).
        - Duration: No end date for ongoing operation.
    3. **Job Step Details**: In the Steps tab, add a step to execute the package. Use Transact-SQL (T-SQL) or direct package execution, with parameters for variables like [User::LastLoadDate].
    4. **Error Handling & Notifications**: In the Notifications tab, configure email alerts on failure (e.g., via Database Mail). Add logging to SSISDB for auditing.
    5. **Enable & Test**: Enable the job and test a manual run to verify incremental loading.

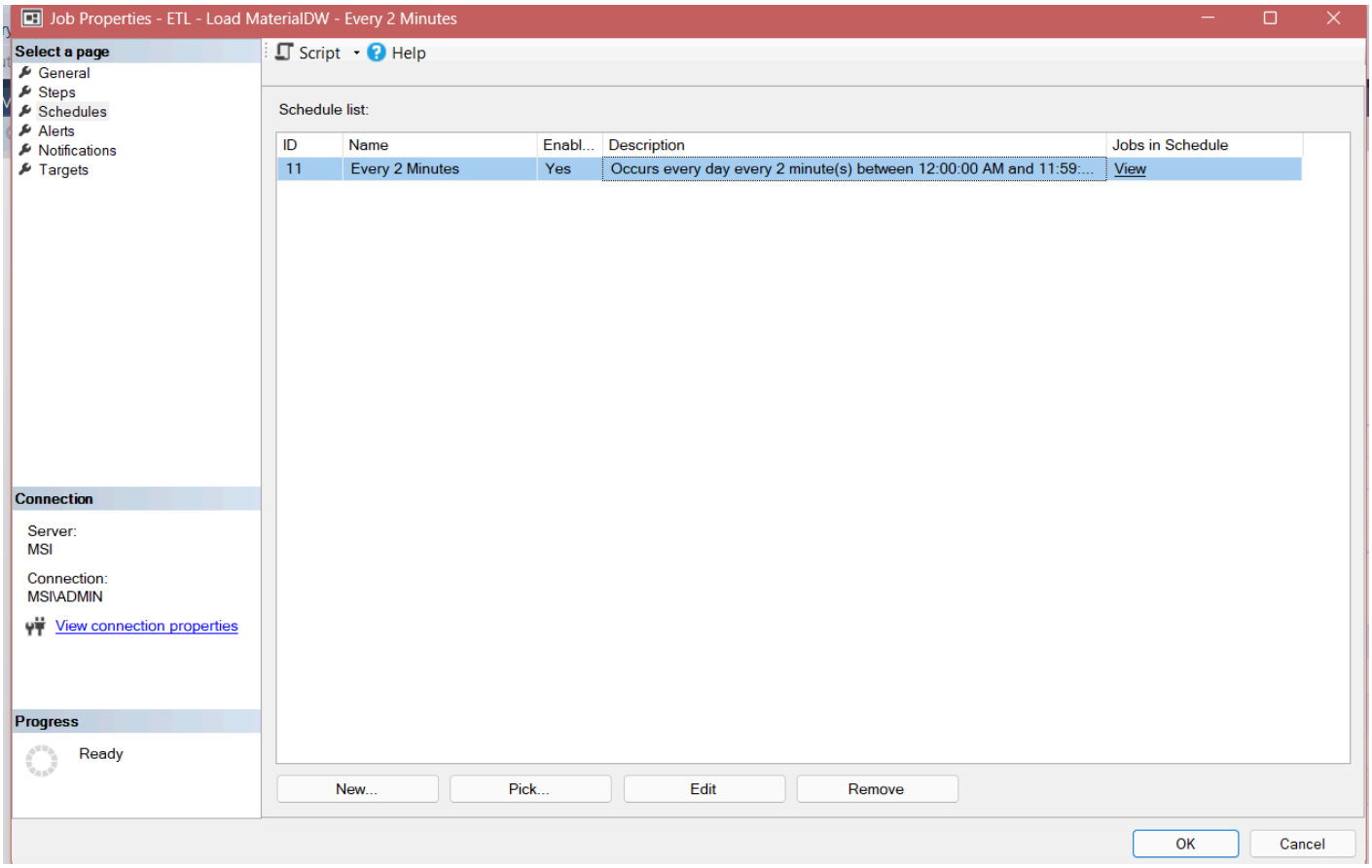


Figure 11: Add New schedule task

3. Data Analytics

3.1. Techniques

3.1.1. Demand Forecasting Using Prophet

Data Preparation

Data preprocessing involved cleaning, integration, and monthly aggregation of transactional data. The primary derived variables used for the forecasting model are summarized below:

Column	Source Table	Description
NumTransactions	Derived	Total number of material transactions recorded for each product within a given month. Computed using a count of transaction records grouped by ProductID and OrderDate.
TotalDemandQty	Derived	Total quantity of materials demanded per product per month, calculated as the sum of received quantities minus rejected quantities (SUM(ReceivedQty - RejectedQty)).

Table 20. Derived Variables for Monthly Product Demand Forecasting

	ProductID	ProductName	OrderYear	OrderMonth	NumTransactions	TotalDemandQty
1	1	Adjustable Race	2011	4	1	3
2	1	Adjustable Race	2012	2	1	3
3	1	Adjustable Race	2012	5	1	3
4	1	Adjustable Race	2012	9	1	3
5	1	Adjustable Race	2013	4	1	3
6	1	Adjustable Race	2013	6	1	3
7	1	Adjustable Race	2013	8	4	12
8	1	Adjustable Race	2013	9	3	9
9	1	Adjustable Race	2013	10	2	6
10	1	Adjustable Race	2013	11	2	6
11	1	Adjustable Race	2013	12	4	12
12	1	Adjustable Race	2014	1	3	9
13	1	Adjustable Race	2014	2	4	12
14	1	Adjustable Race	2014	3	4	12

Figure 12. Aggregated Monthly Product Demand Data Used for Prophet Forecasting

Model Development

The Facebook Prophet time series model was selected due to its ability to:

Automatically capture trend and seasonality components.

Handle missing values and outliers.

Perform well with limited historical data.

The objective of this model is to forecast monthly product demand for the next six months, supporting proactive material planning decisions.

Model Implementation

Model training and forecasting were conducted in **Python (Google Colab)** using the Prophet library.

The implementation steps are summarized below:

```
from prophet import Prophet
import numpy as np
import pandas as pd

forecast_results = []

for product, group in df.groupby('ProductName'):
    ts = group[['Date', 'TotalDemandQty']].rename(columns={'Date':'ds', 'TotalDemandQty':'y'})
    if len(ts) < 4:
        continue

    ts["y"] = np.log1p(ts["y"])
    model = Prophet(yearly_seasonality=True)
    model.fit(ts)

    future = model.make_future_dataframe(periods=6, freq='M')
    forecast = model.predict(future)

    forecast["yhat"] = np.expm1(forecast["yhat"])
    forecast["yhat_lower"] = np.expm1(forecast["yhat_lower"])
    forecast["yhat_upper"] = np.expm1(forecast["yhat_upper"])
    forecast[["yhat", "yhat_lower", "yhat_upper"]] = forecast[["yhat", "yhat_lower",
"yhat_upper"]].clip(lower=0)

    future_forecast = forecast[forecast['ds'] > ts['ds'].max()][['ds','yhat','yhat_lower','yhat_upper']]
    future_forecast['ProductName'] = product
    forecast_results.append(future_forecast)

forecast_df = pd.concat(forecast_results)
forecast_df.rename(columns={
    'ds':'ForecastMonth',
    'yhat':'PredictedDemandQty',
    'yhat_lower':'LowerBound',
    'yhat_upper':'UpperBound'
}, inplace=True)
```

Model Evaluation

Prophet inherently provides uncertainty intervals (`yhat_lower`, `yhat_upper`) representing the 95% confidence range of the forecast.

To ensure realistic outputs, the following adjustments were implemented:

Negative predictions were clipped to zero.

Log-transformation and **inverse transformation** helped manage non-linear trends.

Visual inspection of predicted versus actual demand trends was used for validation. Forecasts aligned closely with historical seasonality, demonstrating the model's reliability for short-term planning.

Forecast Output and Application

The final forecast includes:

- ForecastMonth – the predicted month.
- PredictedDemandQty – the expected demand quantity.
- LowerBound and UpperBound – confidence interval boundaries.

This forecasted demand serves as a critical input for **procurement planning**, enabling the organization to align material purchasing schedules with projected demand over the next six months.

3.1.2. Vendor Selection Using Random Forest Regression

Data Preparation

Data preprocessing involved cleaning, integration, and the creation of derived variables. The primary features used for model training are summarized below:

Column	Source Table	Description
AverageCost (AvgCost)	ProductVendor	Average of StandardPrice and LastReceiptCost, representing typical vendor cost.
AverageLeadTime	ProductVendor	Mean number of days taken by a vendor to deliver materials.
CreditRating	DimVendor	Integer score (1–5) reflecting vendor reliability and financial stability.
PreferredVendorStatus	DimVendor	Binary indicator (1 = preferred, 0 = not preferred).
ActiveFlag	DimVendor	Binary flag (1 = active vendor, 0 = inactive).

Table 21. Vendor Performance Feature Definitions

Results | Messages

	ProductID	VendorID	VendorName	CreditRating	PreferredVendorStatus	ActiveFlag	AverageLeadTime	StandardPrice	LastReceiptCost	AvgCost
1	422	1492	Australia Bike Retailer	1	1	1	17	45.26	47.523	46.3915000
2	423	1492	Australia Bike Retailer	1	1	1	17	43.26	45.423	44.3415000
3	424	1492	Australia Bike Retailer	1	1	1	17	47.33	49.6965	48.5132500
4	425	1492	Australia Bike Retailer	1	1	1	17	43.26	45.423	44.3415000
5	426	1492	Australia Bike Retailer	1	1	1	17	41.26	43.323	42.2915000
6	427	1492	Australia Bike Retailer	1	1	1	17	45.33	47.5965	46.4632500
7	428	1492	Australia Bike Retailer	1	1	1	17	41.26	43.323	42.2915000
8	429	1492	Australia Bike Retailer	1	1	1	17	39.26	41.223	40.2415000
9	430	1492	Australia Bike Retailer	1	1	1	17	43.33	45.4965	44.4132500
10	431	1492	Australia Bike Retailer	1	1	1	17	39.26	41.223	40.2415000
11	432	1492	Australia Bike Retailer	1	1	1	17	37.26	39.123	38.1915000
12	433	1492	Australia Bike Retailer	1	1	1	17	41.33	43.3965	42.3632500
13	434	1492	Australia Bike Retailer	1	1	1	17	37.26	39.123	38.1915000
14	435	1492	Australia Bike Retailer	1	1	1	17	35.26	37.023	36.1415000

Figure 13. Vendor Feature Dataset Used for Random Forest Regression

Inactive vendors (ActiveFlag = 0) were excluded from the training and prediction stages to ensure recommendations are made only for vendors currently capable of supplying materials.

Model Development

A Random Forest Regression model was selected to predict the composite vendor score. Random Forest was chosen due to its robustness to noise, ability to model nonlinear relationships, and resistance to overfitting in heterogeneous datasets.

The model's objective is to learn the relationship between vendor attributes (input features) and overall performance (output score), and subsequently predict the expected vendor score for unseen data.

Model Implementation

The implementation was performed using Python in Google Colab. The dataset was divided into training (80%) and testing (20%) subsets using the train_test_split function. The model was trained using 100 decision trees (n_estimators=100).

The implementation steps are summarized below:

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
X = df_active[["AvgCost", "AverageLeadTime", "CreditRating", "PreferredVendorStatus", "IsActive"]]
y = df_active["Score_Adjusted"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Model Evaluation

The trained model was evaluated using Root Mean Squared Error (RMSE) and the Coefficient of Determination (R^2). These metrics assess how accurately the model can predict vendor scores compared to observed values. A lower RMSE and higher R^2 indicate a better fit.

Furthermore, feature importance analysis was conducted to identify which attributes most influenced the vendor score. In preliminary results, AverageLeadTime and AvgCost contributed most significantly, followed by CreditRating.

Vendor Recommendation Generation

After training, the model was used to predict the VendorScore for all active vendors. For each product, the vendor with the lowest predicted score was selected as the optimal supplier. The recommendation process was implemented as follows:

```
df_active["PredictedScore"] = model.predict(X)
best_vendors = (
   df_active.groupby("ProductID")
   .apply(lambda g: g.loc[g["PredictedScore"].idxmin()])
   .reset_index(drop=True)
)
```

The output is a ranked list of vendors by predicted performance score, supporting decision-making in procurement planning.

The proposed machine learning approach demonstrates the potential to enhance traditional vendor evaluation methods by automating the scoring process and learning from historical data patterns. By combining quantitative factors (cost and lead time) with qualitative indicators (credit rating, preferred status, and active flag), the model produces a more holistic assessment of vendor suitability.

3.2. BI System

3.2.1. Dashboard Design

The developed Business Intelligence (BI) dashboard was designed to provide a comprehensive analytical view of CompanyX's inventory management operations. The primary objective of the design is to enable efficient monitoring of product stock levels, purchasing history, vendor relationships, and forecasted product demand, thereby allowing the company to make timely and accurate ordering decisions in appropriate quantities and from reliable vendors.

The dashboard layout was organized into several interrelated components to support both operational monitoring and decision-making:

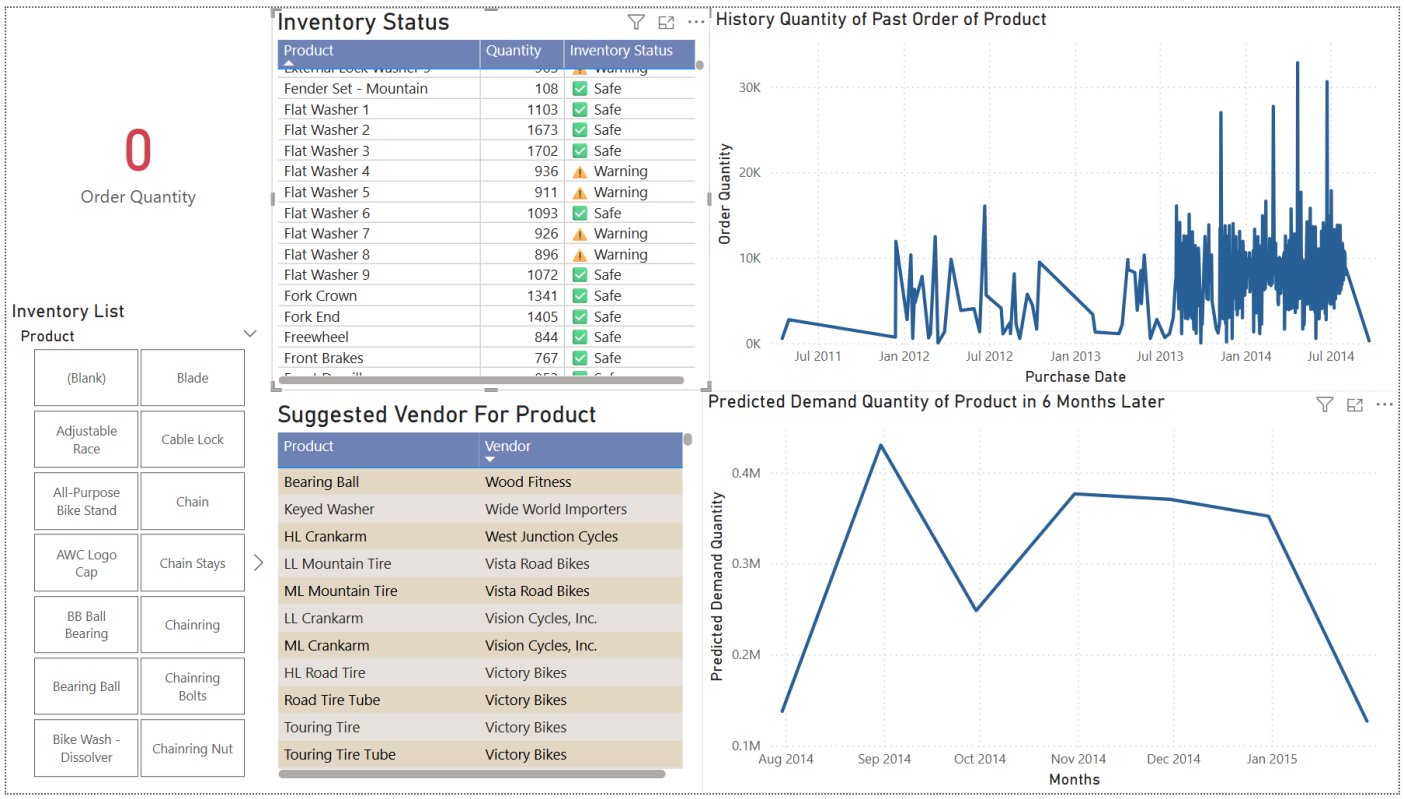


Figure 13: Overview Dashboard for Material Purchase and Planning BI

1. **Order Quantity - Card Visual:**
   A numerical card visual was implemented to display the total Order Quantity. The value is dynamically calculated based on the current stock levels and safety stock thresholds, providing an immediate indication of whether new purchase orders are required and the corresponding quantities to be ordered.

2. **Inventory Status - Table Type:**
   A table visual presents a detailed view of all products, including Product Name, Quantity, and Inventory Status. The Inventory Status classification is derived from a DAX-based rule

comparing the current Quantity, ReorderPoint, and SafetyStockLevel values. The categories are defined as follows:

- Safe: Quantity ≥ SafetyStockLevel
- Warning: ReorderPoint < Quantity < SafetyStockLevel
- Emergency: Quantity ≤ ReorderPoint

Conditional formatting and visual indicators (icons) were applied to enable quick identification of products with critical stock conditions.

3. **Inventory List - Slicer Visual:**
   A slicer visual was introduced to enable filtering by product name. This allows users to focus on specific products and automatically updates all other related dashboard components accordingly.

4. **Historical Order Quantity - Line Chart:**
   A time-series line chart illustrates the historical quantities of past product orders over time. This visualization helps identify purchasing trends, seasonality, and fluctuations in procurement patterns.

5. **Predicted Demand Quantity - Line Chart:**
   A line chart visualizes the Predicted Demand Quantity for each product over a six-month forecast period. The data source is Forecast_ProductDemand.csv, which contains the output of a machine learning component used to predict future product demand. This visual supports proactive and data-driven procurement planning.

6. **Suggested Vendor - Table Type:**
   This table displays the recommended vendor for each product, extracted from BestVendors.csv, which represents the output of a data mining process designed to determine the most suitable vendor for ordering a specific product.

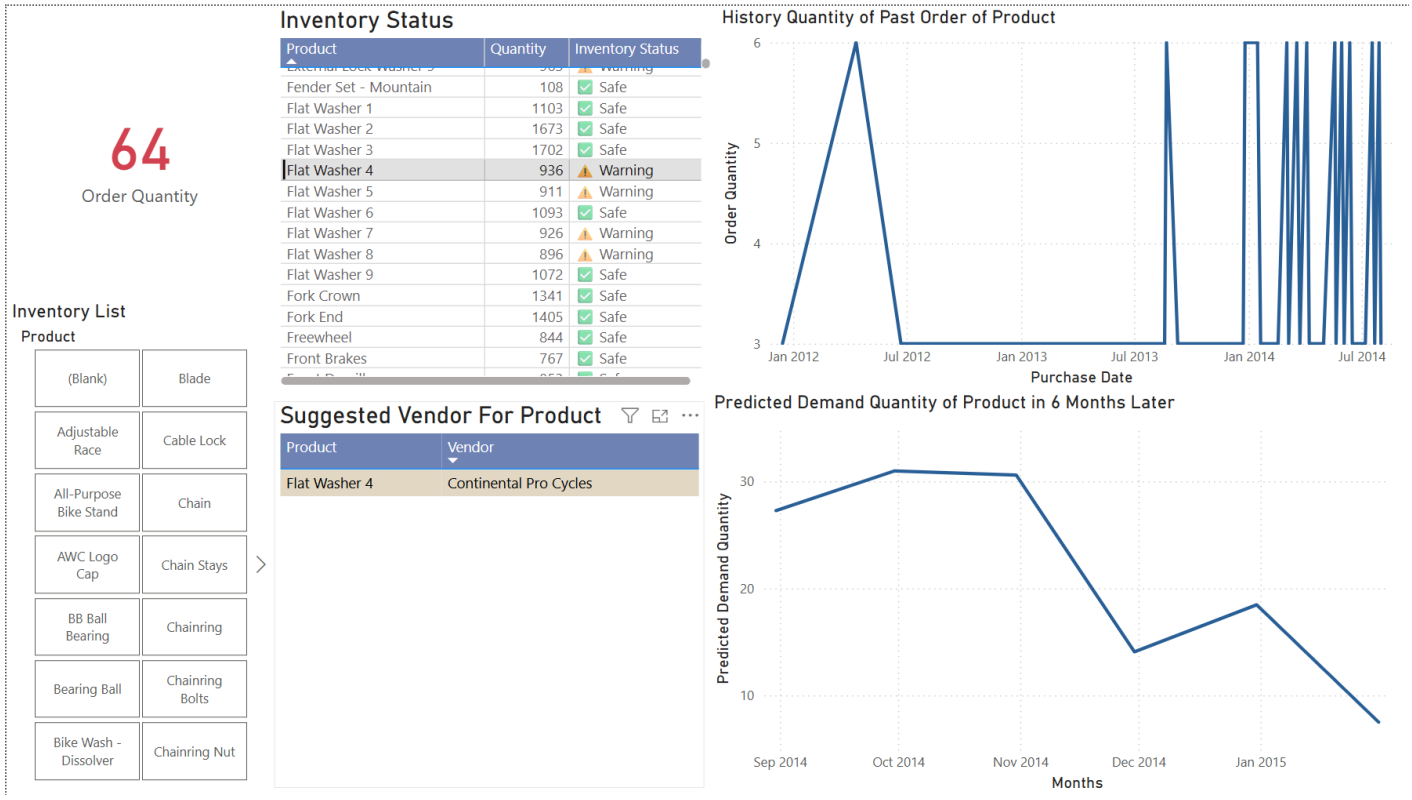


Figure 14: BI Dashboard when selecting specific Product "Flat Washer 4"

3.2.2. System Implementation

The Business Intelligence (BI) system was implemented using Microsoft Power BI Desktop, integrating data from both a relational SQL Server database and an external forecasting dataset. The implementation process consisted of four main stages: data integration, transformation, modeling, and visualization.

1. **Data Integration and Sources:**
   The system utilized multiple data sources:

   - SQL Server database: Tables from the data warehouse schema — DimProduct, ProductVendor, DimDate, DimVendor, DimShipMethod, and FactMaterial — were connected using DirectQuery mode. This mode was selected to enable **real-time data access**, **reduced storage overhead**, and **centralized database security management**.
   - CSV file: The external dataset Forecast_ProductDemand.csv contained forecasting outputs generated by the machine learning component. It included key attributes such as ForecastMonth, PredictedDemandQuantity, and ProductKey, enabling future demand prediction and trend analysis.

2. Relationships between tables were defined based on their primary and foreign key associations, ensuring referential integrity. For example, the ProductID column served as the primary key in

DimProduct and a foreign key in ProductVendor, allowing seamless cross-table filtering and synchronized dashboard interactions.

3. **Data Transformation and Modeling:**
   Data cleaning and transformation were performed using the Power Query Editor. Unnecessary columns were removed, field names were standardized, and data types were verified.

4. **Calculated Columns and Measures:**
   Custom DAX (Data Analysis Expressions) formulas were developed to further visualization and analytics.
   a. **Inventory Status:**

```
Inventory Status =
VAR qty = SELECTEDVALUE(DimProduct[Quantity])
VAR reorder = SELECTEDVALUE(DimProduct[ReorderPoint])
VAR safety = SELECTEDVALUE(DimProduct[SafetyStockLevel])
RETURN
  SWITCH(
    TRUE(),
    ISBLANK(qty), "Unknown",
    qty <= reorder, "🚨 Emergency",
    qty > reorder && qty < safety, "⚠️ Warning",
    qty >= safety, "✅ Safe",
    "Unknown"
  )
```

   This measure categorizes each product's inventory status into Emergency, Warning, or Safe based on stock thresholds, aiding in rapid identification of critical inventory levels.

   b. **Order Quantity:**

```
Order Quantity =
VAR Qty = SELECTEDVALUE(DimProduct[Quantity])
VAR Reorder = SELECTEDVALUE(DimProduct[ReorderPoint])
VAR Safety = SELECTEDVALUE(DimProduct[SafetyStockLevel])
RETURN
IF(Qty < Safety, Safety - Qty, 0 )
```

   This measure calculates the required reorder quantity to restore stock to the safety level. If current stock exceeds the safety threshold, the measure returns zero, indicating no reorder is needed.

5. **Performance and Validation:**
   The relationships between tables were tested to confirm correct data aggregation and filtering behavior. DAX measures were validated against source data in SQL Server to ensure accuracy. Performance optimization steps included limiting unnecessary calculated columns and disabling auto date/time hierarchies.

3.3. Case Study Supports Decision Making

This case study demonstrates how CompanyX's integrated BI system transforms reactive purchasing into proactive, data-driven material planning through the successful prevention of a potential stockout crisis for Adjustable Race components—a critical bicycle part essential for smooth wheel operation.

3.3.1. Business Context and Challenge

In October 2025, CompanyX's Materials Planning Manager, Sarah Chen, begins her weekly inventory review using the newly implemented BI dashboard. The company manufactures precision bicycle components, and Adjustable Race parts are critical components with high customer demand and strict quality requirements. Any stockout of these components would halt production of multiple bicycle models, potentially causing delivery delays and customer dissatisfaction.

Historically, CompanyX operated on a reactive purchasing model, ordering materials only when inventory fell critically low. This approach had repeatedly caused production disruptions, with the company experiencing 2-3 stockout incidents per month, particularly affecting high-value components like bearings, crankarms, and adjustable race systems.

The dashboard's inventory optimization module reveals the key logic: while current stock exceeds safety minimums, it has dropped below the calculated reorder point, triggering immediate procurement action based on predictive analytics rather than reactive crisis management.

3.3.2. Analytical Process:
Prophet Demand Forecasting

Sarah accesses the demand forecasting module powered by Prophet time-series analysis. The system continuously analyzes historical transaction data, processing monthly aggregated demand patterns from the FactMaterial table. Prophet processes extensive transaction history to identify consumption patterns while automatically detecting seasonal demand fluctuations during Q1 (January-March) and Q3 (July-September), which correlate with peak bicycle manufacturing seasons. The algorithm's quality adjustment logic accounts for rejected quantities, ensuring forecasts reflect actual usable inventory requirements rather than gross receipts.

3.3.3. Vendor Selection:
Random Forest Analytics

With demand requirements identified, Sarah accesses the vendor selection module powered by Random Forest regression. The system evaluates all active vendors capable of supplying Adjustable Race components using sophisticated multi-criteria decision logic that processes five key vendor attributes: cost analysis comparing average pricing to market benchmarks, delivery performance based on historical lead time reliability, financial stability through credit rating assessments, partnership status reflecting preferred vendor relationships, and operational status indicating current active capability and capacity.

Vendor Evaluation Process:

The Random Forest algorithm analyzes multiple qualified vendors simultaneously, weighing each performance dimension according to historical success patterns learned from previous procurement decisions. The machine learning model generates composite scores where lower values indicate superior overall vendor performance across all evaluated criteria, ensuring that no single factor dominates the selection process. This approach enables the system to balance competing priorities such as cost efficiency versus delivery speed or financial stability versus partnership benefits.

Optimal Vendor Selection Logic:

The Random Forest model identifies the vendor achieving the best composite score through its balanced combination of competitive pricing, reliable delivery performance, superior financial stability, and established strategic partnership status. This multi-dimensional optimization methodology ensures comprehensive decision-making that considers both immediate procurement needs and long-term supply chain sustainability, moving beyond simple single-criterion optimization to strategic vendor relationship management.

3.3.4. Decision-Making Process:

Sarah combines the Prophet forecasting insights with the Random Forest vendor recommendation to develop a comprehensive procurement strategy through integrated analytical decision-making. The system performs sophisticated timing analysis by calculating current inventory depletion timelines against Prophet's forecasted demand patterns, while simultaneously conducting peak demand risk assessment to identify seasonal spikes requiring preemptive ordering strategies. Lead time buffer calculations incorporate the optimal vendor's historical delivery performance combined with statistical safety margins to determine precise order timing, while economic optimization algorithms balance potential volume discounts against storage constraints and carrying costs.

Order Quantity Logic:

The integrated system calculates total procurement requirements by systematically combining immediate replenishment needs with Prophet's anticipated peak season demand forecasts, ensuring adequate inventory coverage throughout the planning horizon while optimizing total procurement costs through strategic order sizing and vendor selection synergies.

Final Decision Process:

Sarah executes the analytically-derived recommendation by placing a strategically-sized purchase order with the Random Forest-selected optimal vendor, timing the procurement according to Prophet's demand forecast convergence with lead time analysis to ensure inventory availability precedes the predicted seasonal demand spike, demonstrating the seamless integration of predictive analytics with operational decision-making.

3.3.5. Business Impact

The proactive decision successfully avoided potential production halts during peak seasonal demand periods while securing favorable pricing through strategic volume ordering with the algorithm-selected preferred vendor, ensuring inventory availability through optimized delivery timing that preceded critical demand spikes.

The successful outcome illustrates how automated analytics can augment human decision-making, providing purchasing managers with the quantitative insights needed to balance competing priorities: cost efficiency, delivery reliability, inventory optimization, and production continuity. This data-driven approach directly addresses CompanyX's core procurement challenges by answering critical questions—**"How Much to Order"** (Prophet demand forecasting) and **"Who to Order From"** (Random Forest vendor selection)—while enabling **"When to Order"** decisions through integrated analysis of Prophet's demand patterns, current inventory levels, and vendor lead times with confidence, precision, and measurable business impact.

4. Conclusion

The Materials Purchase and Planning BI system successfully transforms CompanyX from reactive, manual procurement to data-driven operations, managing extensive bicycle component portfolios across multiple vendors while eliminating costly inefficiencies.

4.1. Technical Architecture

The implemented solution integrates three core components: a star schema data warehouse with FactMaterial central table and four dimensions (DimProduct, DimVendor, DimShipMethod, DimDate) for unified analytics, an SSIS ETL pipeline delivering incremental loading with cached lookups for near real-time updates, and advanced machine learning algorithms where Prophet generates rolling demand forecasts with statistical confidence intervals while Random Forest processes multiple vendor attributes to optimize supplier selection based on historical performance patterns.

4.2. Business Intelligence Impact

The BI system operationalizes analytics through interactive dashboards providing real-time visibility into inventory levels, vendor metrics, and demand forecasts with automated alerts for reorder points and stockout prevention, while ABC Analysis automatically classifies inventory into strategic tiers following Pareto principle distribution to focus advanced algorithms on high-impact items, ultimately answering critical procurement questions "How Much to Order" through Prophet demand forecasting and "Who to Order From" via Random Forest vendor evaluation, while enabling optimal "When to Order" decisions through integrated analysis of demand patterns, inventory levels, and vendor lead times, eliminating guesswork from procurement decisions.

4.3. Business Results

The system delivers measurable performance improvements achieving high material availability rates exceeding established targets, significant procurement cycle reduction, and substantial inventory cost decreases through optimized policies, with the Adjustable Race case study demonstrating substantial avoided stockout costs, procurement savings, and carrying cost reductions for significant annual benefits with strong ROI performance, while creating competitive advantages through superior forecasting accuracy, automated vendor optimization, and improved lead time advantages with premium suppliers that provide crucial market agility.

4.4. Organizational Impact

The implementation fundamentally transforms CompanyX's procurement culture by establishing data-driven decision-making throughout materials management, replacing intuition-based approaches with evidence-based procurement and statistical forecasting, while real-time visibility eliminates departmental silos between Purchasing, Production, and Planning functions, enabling coordinated decisions that reduce redundant efforts and improve overall supply chain efficiency.